



Applied ML

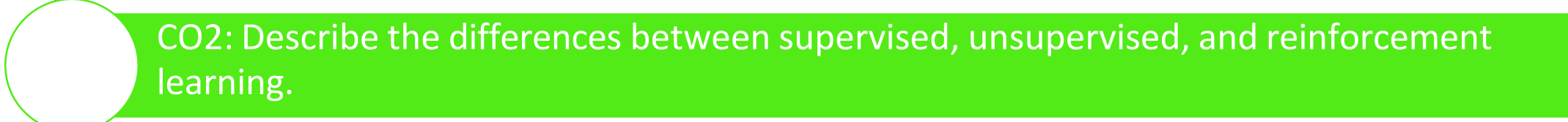
Prerequisite(s): Elements of AI

Total Units to be Covered: 6	Total Contact Hours: 48
-------------------------------------	--------------------------------

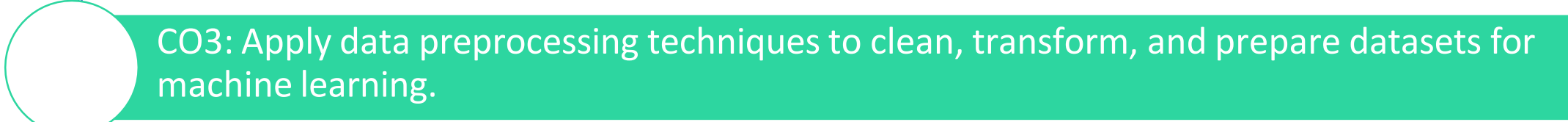
Course Outcomes



CO1: Recall and define key machine learning concepts, terminologies, and algorithms.



CO2: Describe the differences between supervised, unsupervised, and reinforcement learning.



CO3: Apply data preprocessing techniques to clean, transform, and prepare datasets for machine learning.

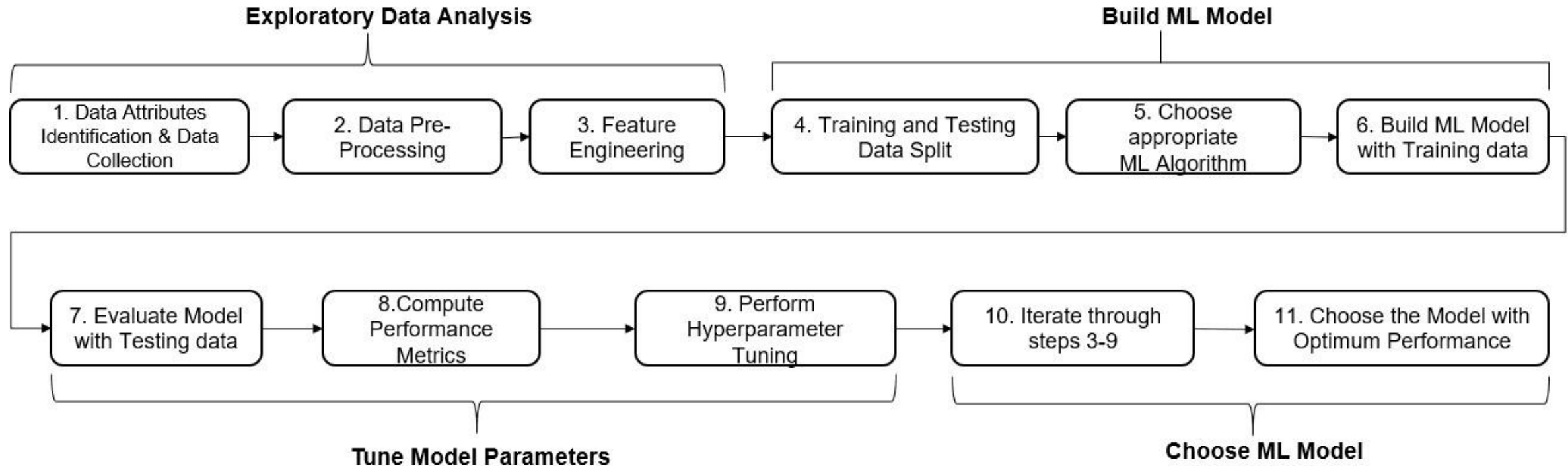


CO4: Apply, compare, and contrast the strengths and weaknesses of different machine learning algorithms.

Unit IV: Data Pre-processing 10 Lecture Hours

Data Cleaning: handling Missing Data, Handling Outlier, Data Transformation: Feature Scaling, Feature Encoding, Feature Engineering, Data Reduction: Dimensionality reduction technique, feature selection, Data Splitting: Cross validation techniques, Handling imbalanced data: Oversampling techniques, under sampling techniques.

Machine Learning Model Development Lifecycle



Exploratory Data Analysis - EDA

- **Goal:**

- The primary goal of EDA is to visually and statistically explore the dataset to understand the data and discover patterns, relationships, and potential insights in the dataset.

- **Output:**

- The primary output of EDA is knowledge and insights about the dataset, including visualizations, summary statistics, and potential hypotheses.

- **EDA Step 1 Activities:**

Involves activities such as univariate analysis, bivariate analysis, and multivariate analysis. These types of analyses help understand patterns, trends, and relationships within a dataset.

Exploring correlations between variables.

Identifying outliers.

Data visualization:
Creating plots, charts, and graphs to visualize data distributions, relationships, and patterns.

Summary statistics:
Calculating basic statistics like mean, median, standard deviation, and quartiles to describe the data.

Example – Iris Dataset

iris setosa



petal sepal

iris versicolor



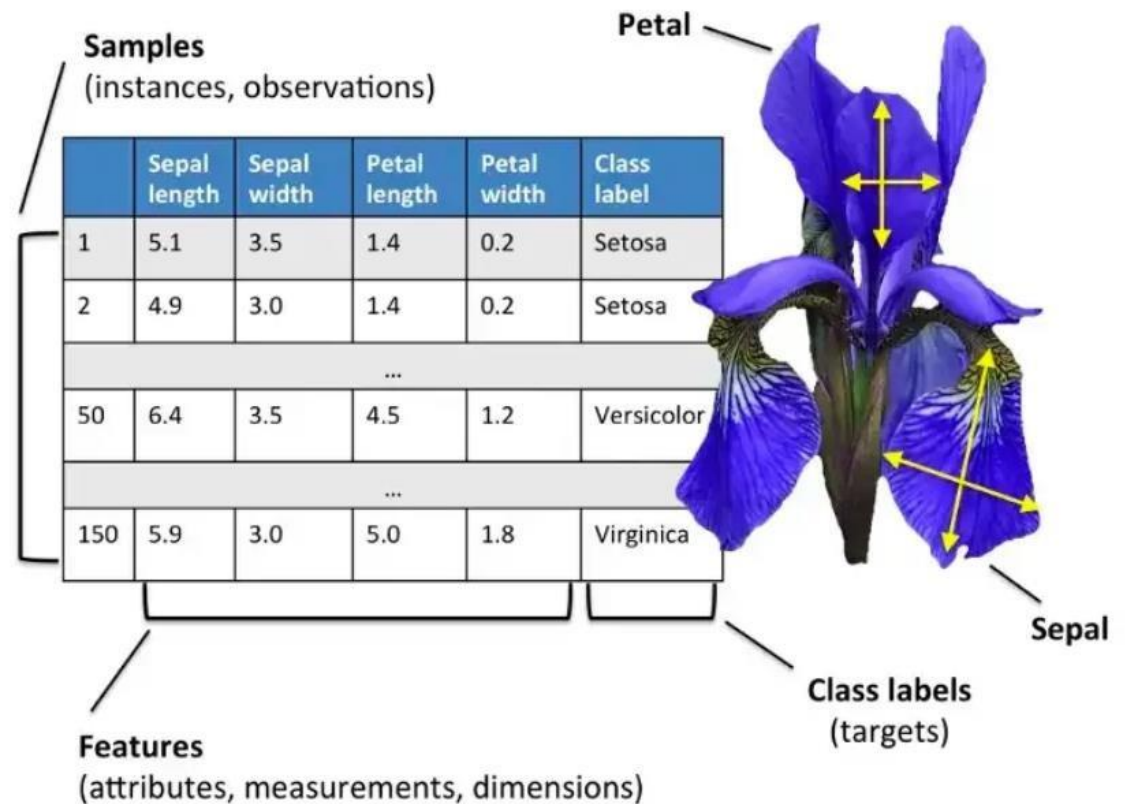
petal sepal

iris virginica



petal sepal

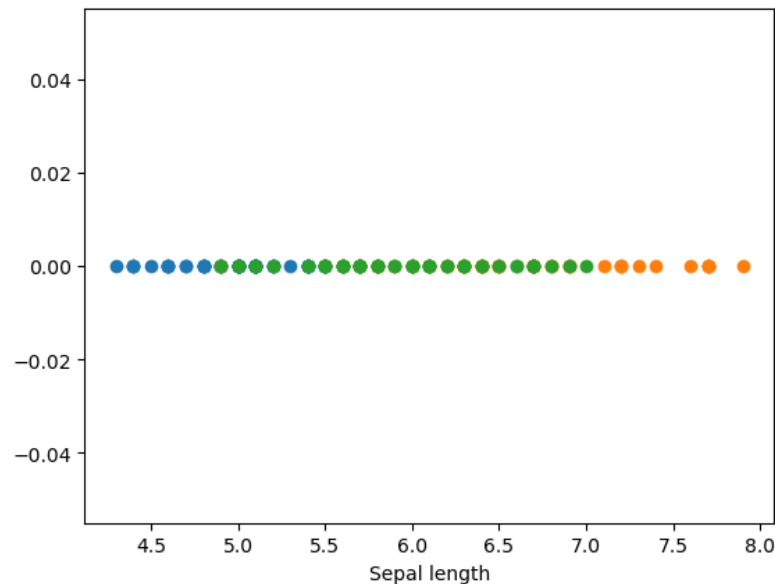
It consists of 150 observations of iris flowers, including the sepal and petal length and width for each flower, as well as the species of the flower.



EDA Step 1: Uni, Bi, Multivariate Analysis

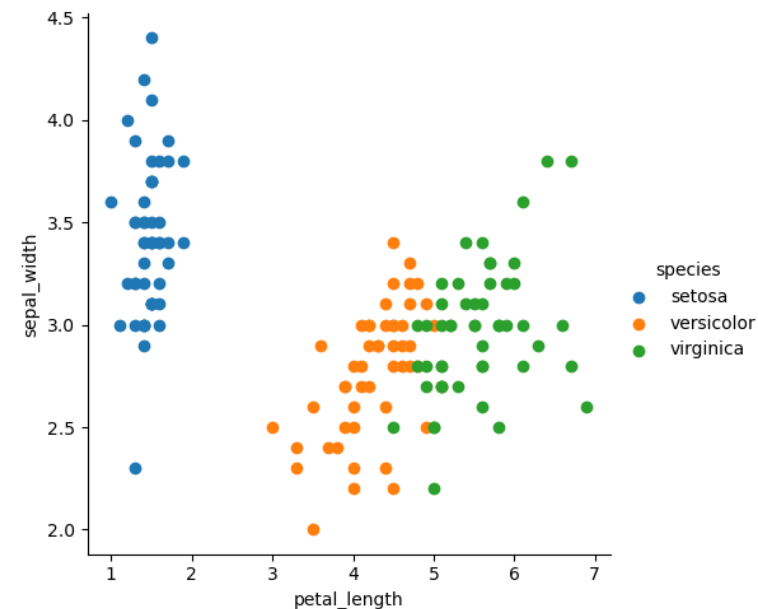
Univariate Analysis

Univariate analysis involves the examination of a single variable, focusing on its distribution and summary statistics.



Bivariate Analysis

Bivariate analysis involves the examination of the relationship between two variables. We also try to see if we can classify through two features. E.g., in the Iris data set, analysing how petal length and sepal width for classification.

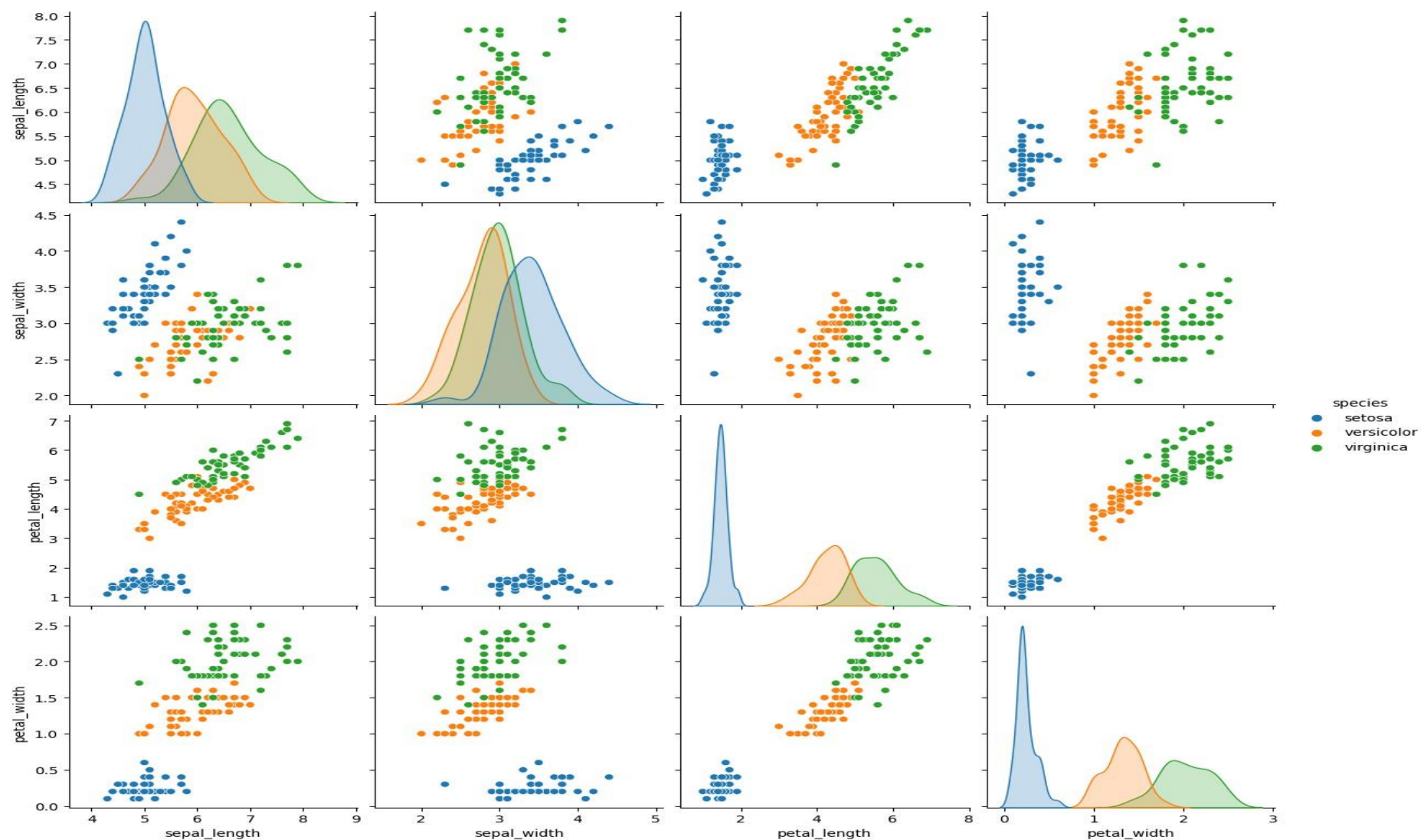


sepal_length sepal_width petal_length petal_width species(setosa, virginica, versicolor)

sepal_length sepal_width petal_length petal_width species (setosa, versicolor, virginica)

Multivariate Analysis

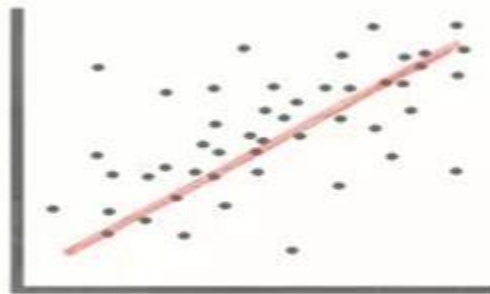
Multivariate analysis involves the examination of more than two variables simultaneously.



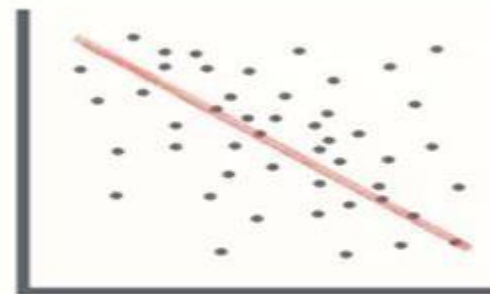
EDA Step 2: Exploring correlations between variables

Correlation Coefficient

- The correlation coefficient is a statistical measure that quantifies the strength and direction of a linear relationship between two variables.
- A correlation coefficient greater than zero indicates a positive relationship, while a value less than zero signifies a negative relationship.
- A value close to zero indicates a weak relationship between the two variables being compared.



Positive Correlation



Negative Correlation



No Correlation

WHAT IS A CORRELATION COEFFICIENT?



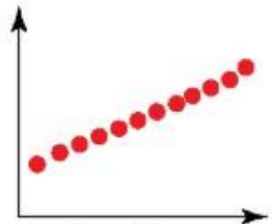
A correlation coefficient is a number that is used to describe the strength of a relationship between two variables.

These numbers range from -1 to +1, with zero describing no correlation at all.

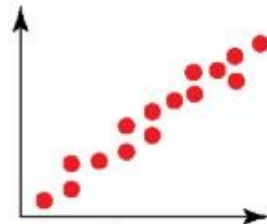
If two variables have a correlation coefficient of 1, they have a perfectly linear correlation.



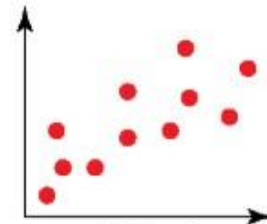
When one goes up, the other will go up in the same proportion, and when one goes down, so will the other.



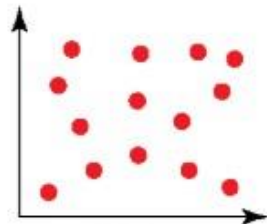
Perfect
Positive
Correlation



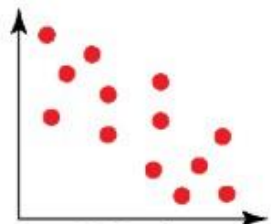
Strong
Positive
Correlation



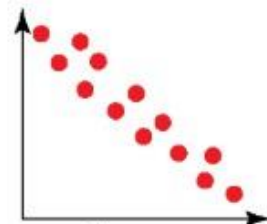
Weak
Positive
Correlation



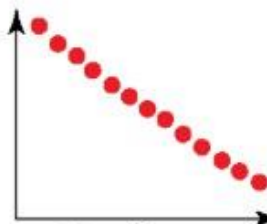
No
Correlation



Weak
Negative
Correlation



Strong
Negative
Correlation



Perfect
Negative
Correlation

Covariance

Covariance gives the joint relationship between two random variables.

It is calculated using the following formula:

$$\text{Cov}(X, Y) = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{n}$$

where,

X, Y = random variables

X_i = data value of x

Y_i = data value of y

$\bar{X}$ = mean of all values of X

$\bar{Y}$ = mean of all values of Y

n = Total number of values of X or Y

$$\rho_{xy} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}$$

ρ_{xy} = Pearson's product-moment correlation coefficient

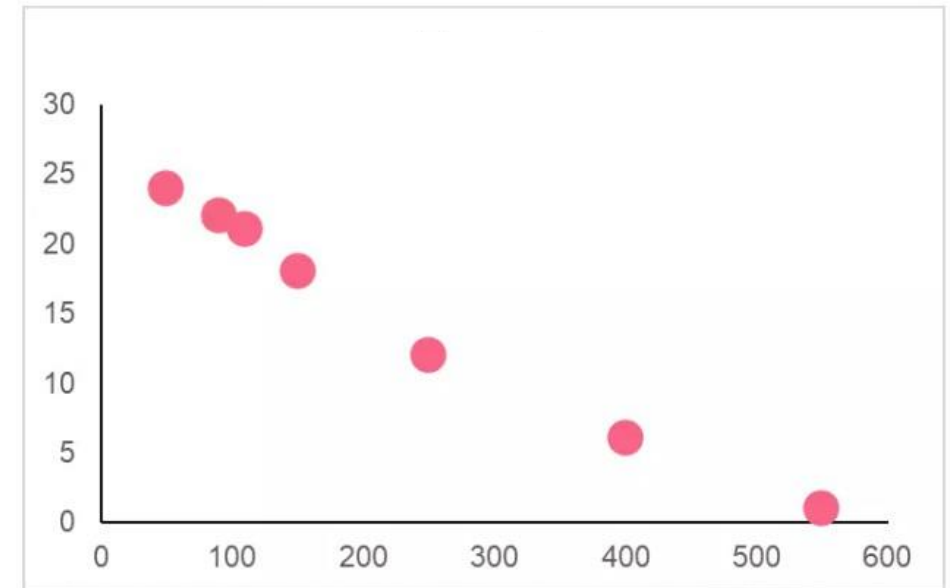
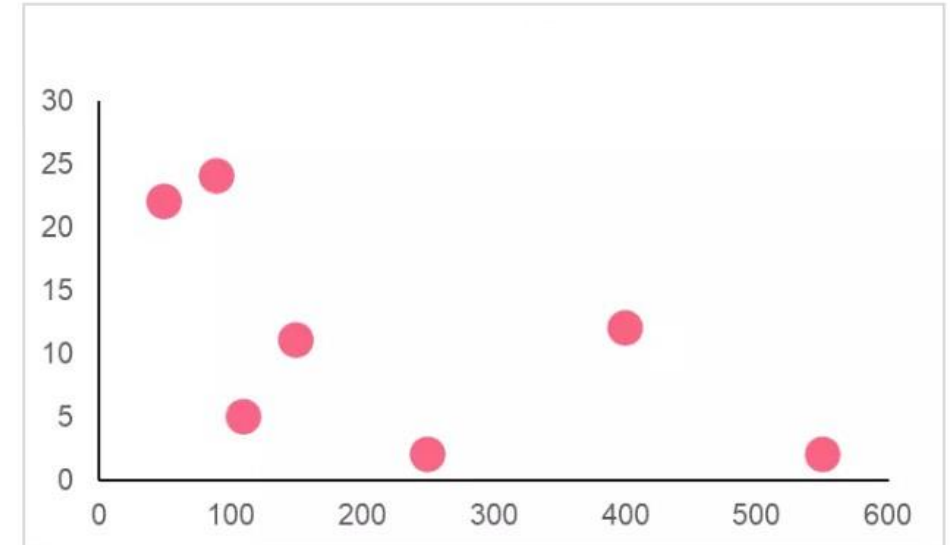
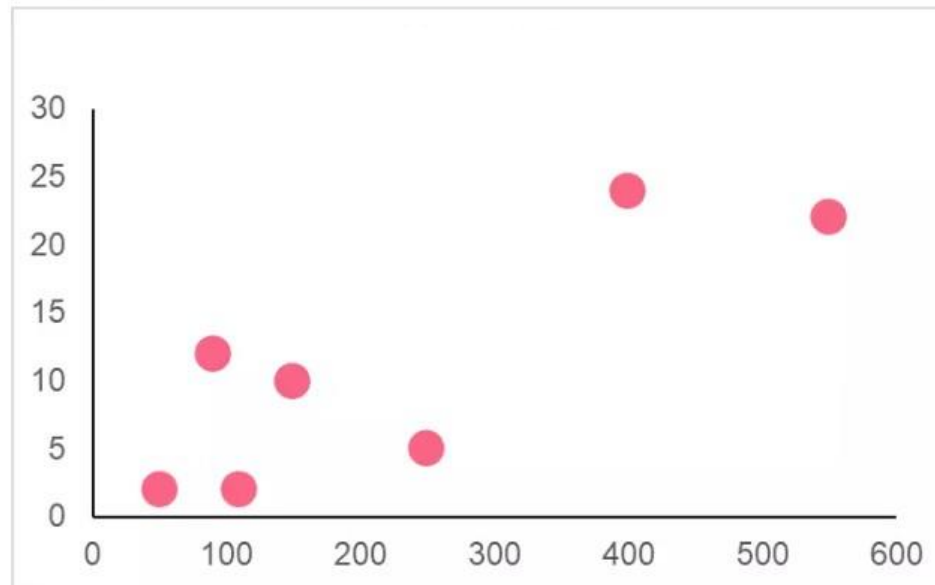
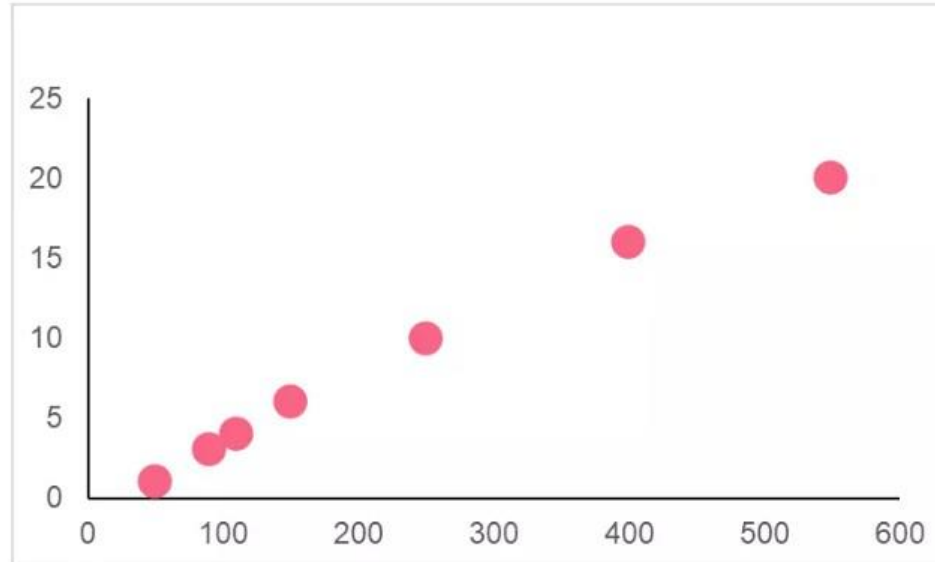
$\text{Cov}(x, y)$ = covariance of variables x and y

σ_x = standard deviation of x

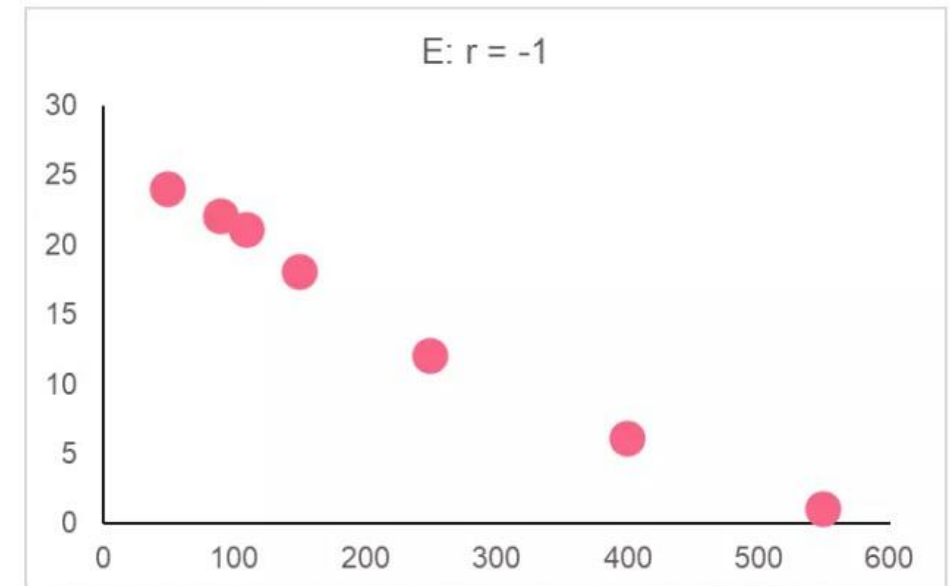
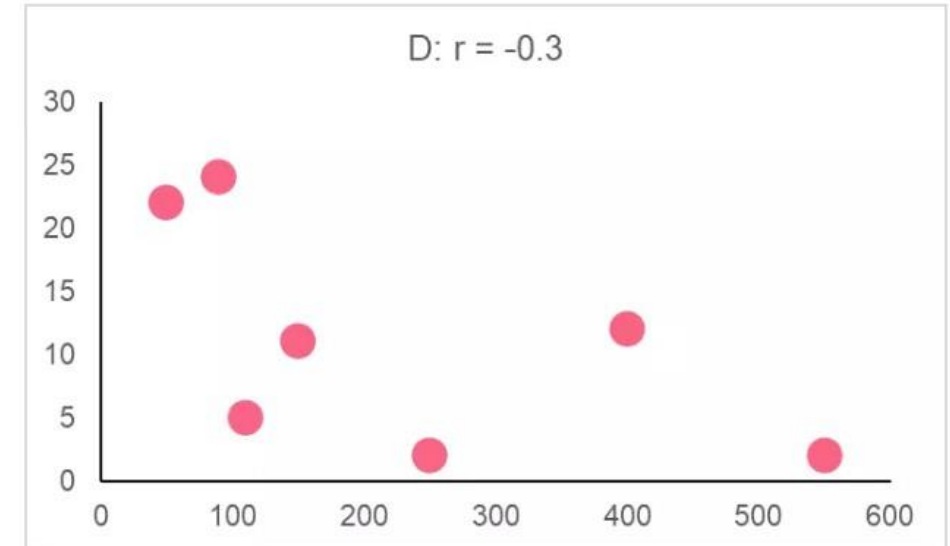
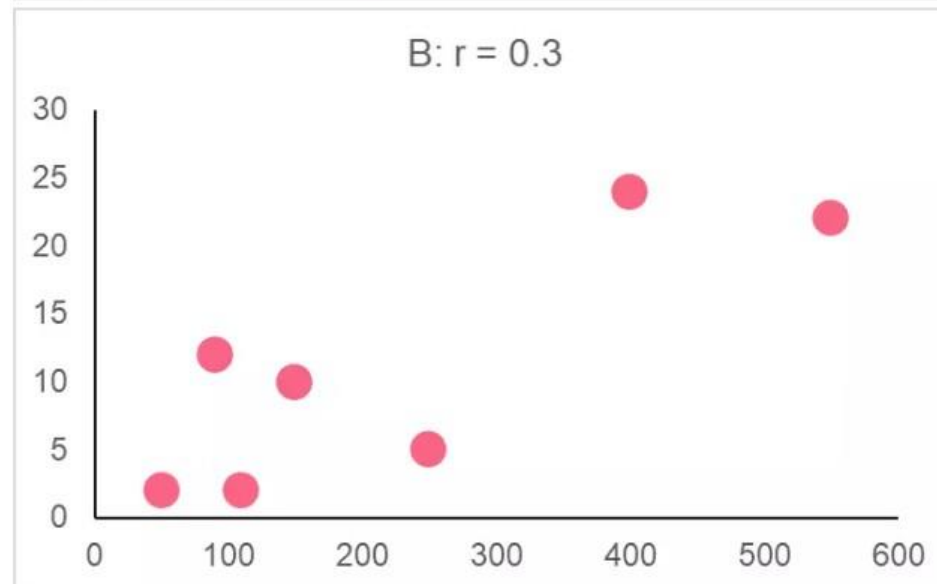
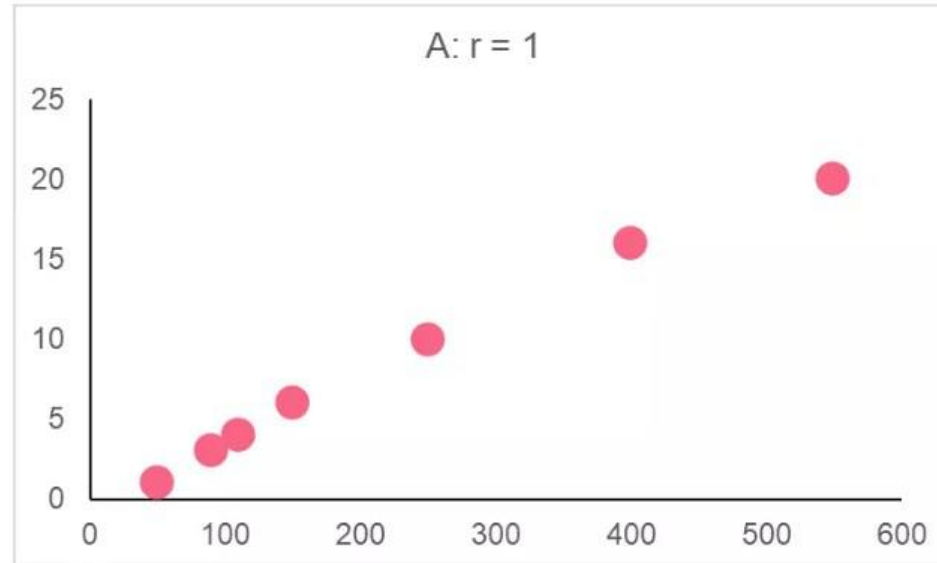
σ_y = standard deviation of y

What Is Considered a Strong Correlation Coefficient?

- Generally, the closer a correlation coefficient is to 1.0 (or -1.0), the stronger the relationship between the two variables is said to be.
- While there is no clear boundary to what **makes a "strong" correlation, a coefficient above 0.75 (or below -0.75) is considered a high degree of correlation, while one between -0.3 and 0.3 is a sign of weak or no correlation.**
- **Multicollinearity** is a statistical phenomenon that occurs when two or more independent variables are highly correlated with each other.
- Highly correlated features provide redundant information about the target variable. **Keeping both features may not add much value to the model** and choosing one might be more efficient.



https://www.superprof.co.uk/resources/academic/maths/statistics/inference/problems-of-correlation-and-regression.html#chapter_problem-1



https://www.superprof.co.uk/resources/academic/maths/statistics/inference/problems-of-correlation-and-regression.html#chapter_problem-1

EDA- Outliers

What is Outlier?

- An outlier is an observation that significantly deviates from the rest of the data points in a dataset.
- It is a data point that lies an abnormal distance away from other values in a random sample from a population.
- The presence of outliers can substantially impact statistical analysis, as they can skew results and lead to incorrect conclusions.

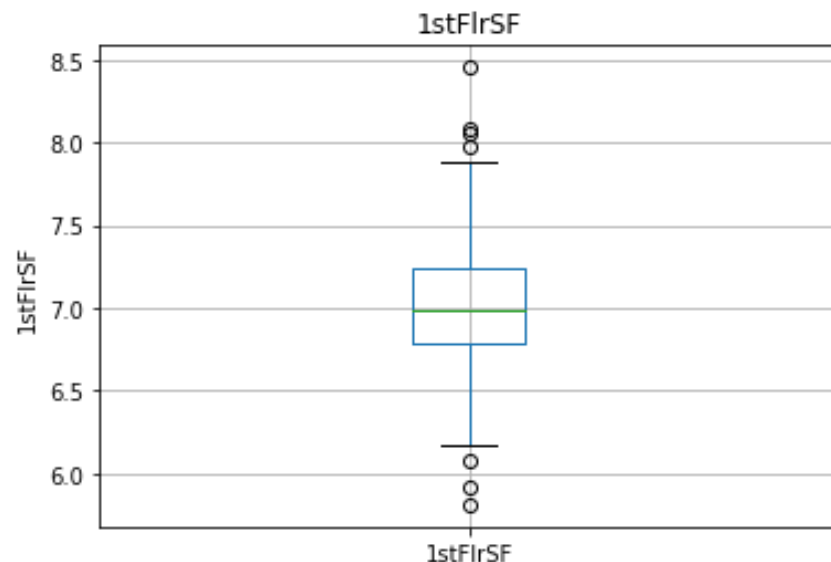
How do we find the presence of an Outliers

- **Understanding the domain and context** of the data is crucial. In some cases, data points that might be considered outliers by statistical methods are actually valid observations based on domain knowledge.

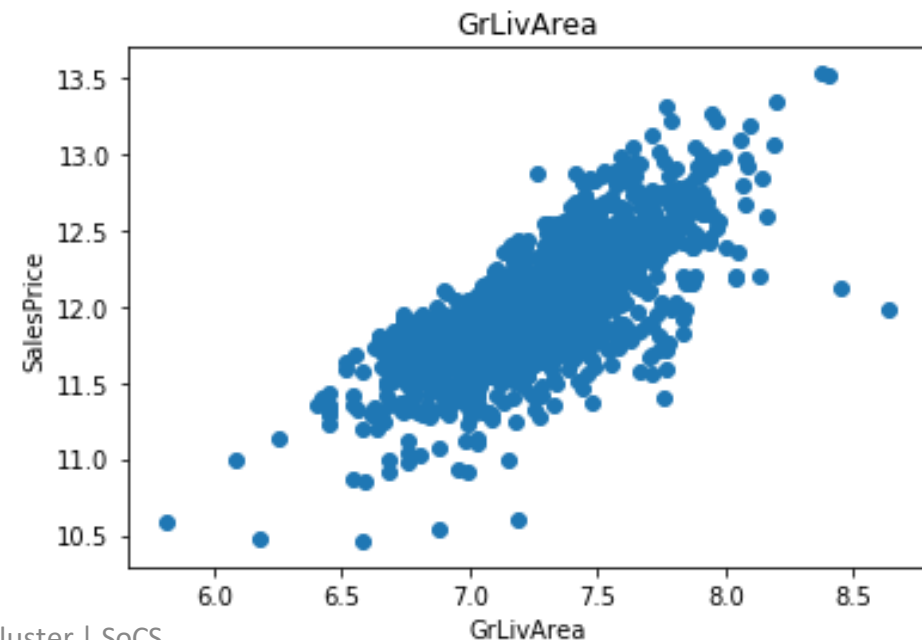
EDA Step 3: Identifying Outliers

Visual Inspection

Box plots are graphical representations that display the distribution of data, including the presence of outliers. Points beyond the whiskers of the box plot are often considered potential outliers.

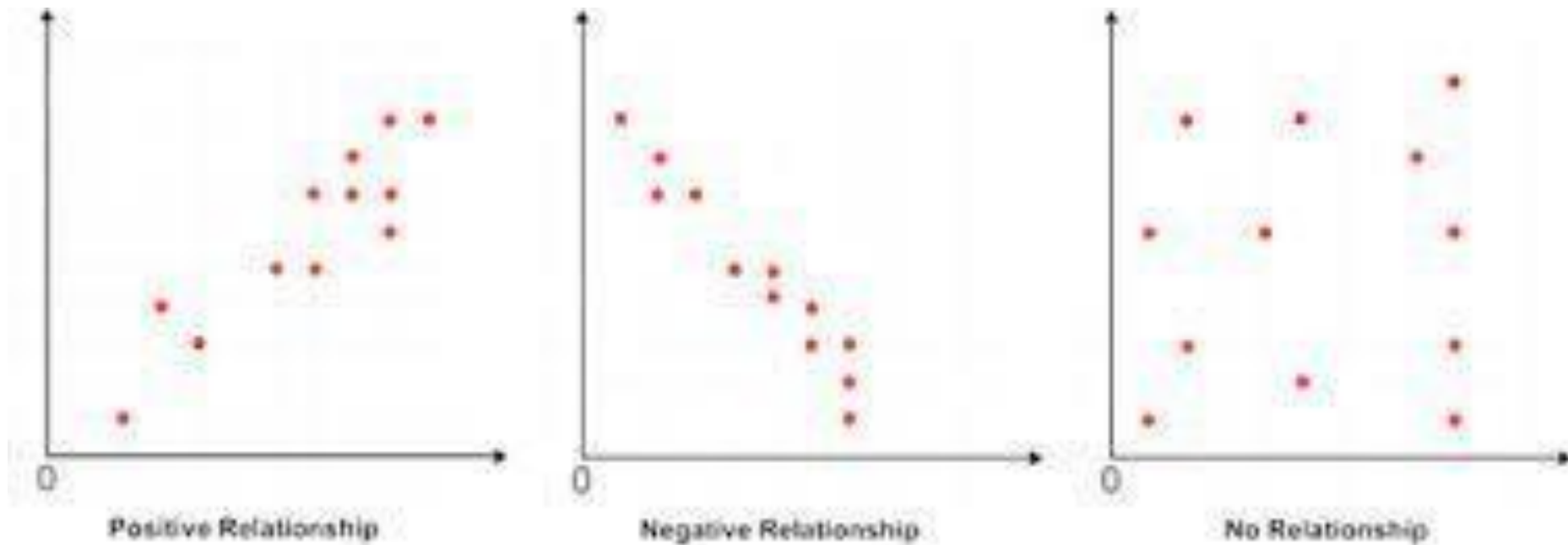


Visualizing the data using **scatter plots** can reveal points far from the data's main cluster. Outliers may appear as individual points with a significant deviation from the general pattern.



• Scatter Plots

- **Scatter plots visualize relationships between two continuous variables.**
- Identify patterns, trends, and correlations between variables. Clusters of points may suggest subgroups or patterns in the data.
- Scatter plots help assess the correlation, whether it's positive, negative, or nonexistent.

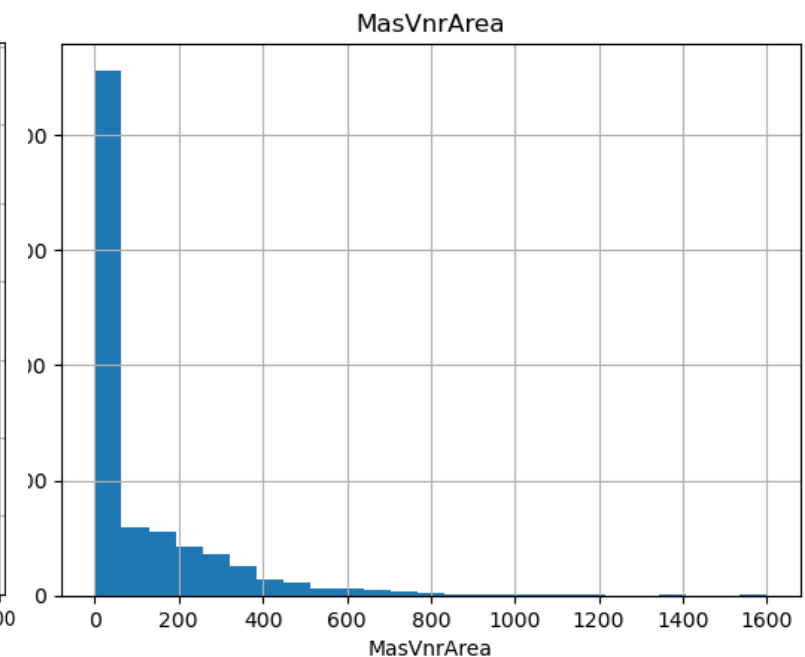
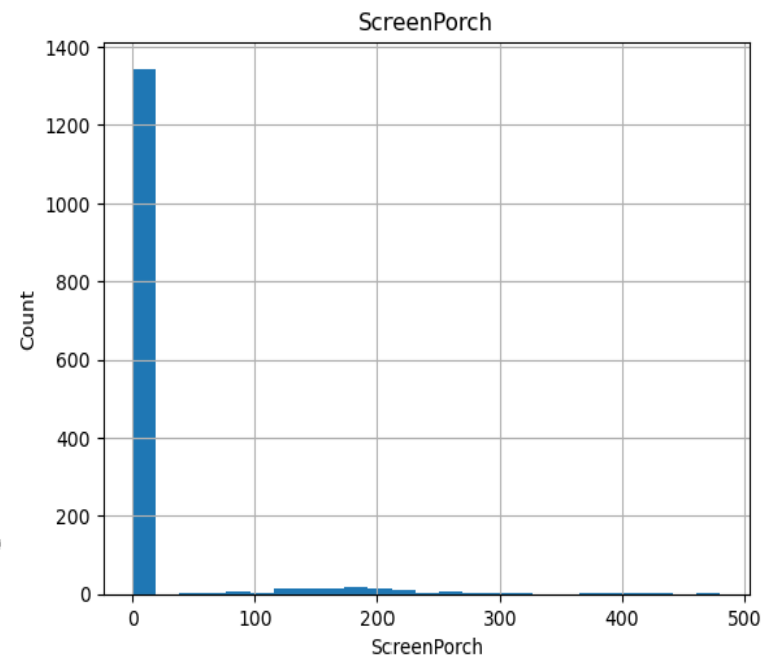
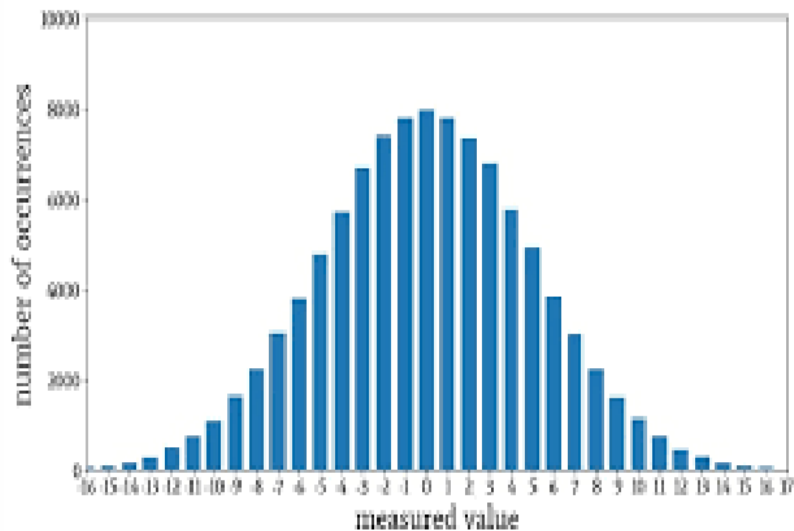


• Line Charts

- Line charts show how a variable changes over time.
- Identify trends, seasonality, or cyclical patterns in time series data.

Histograms

- Histograms provide a **visual representation of the distribution of a single variable.**
- They help identify the data's central tendency, spread, and shape, including whether it follows a normal distribution or exhibits skewness.



How do we find the presence of an Outliers

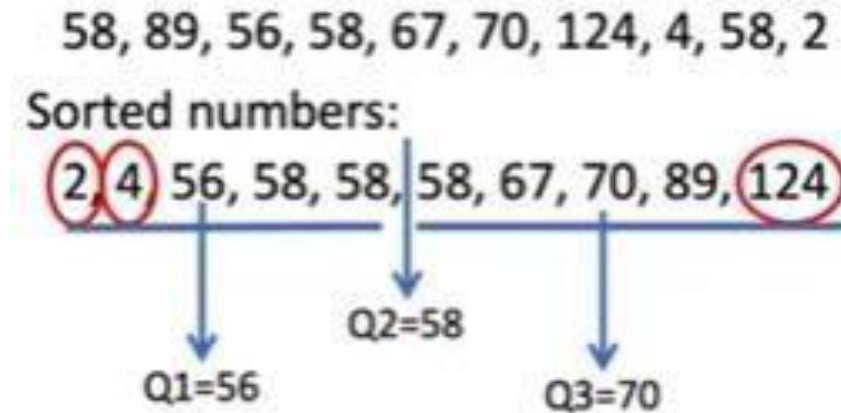
Statistic Methods

The IQR ranges between the first quartile (Q1) and the third quartile (Q3). Data points beyond a certain range from the quartiles are considered outliers.

$$\text{IQR} = Q3 - Q1$$

$$\text{Upper Bound} = Q3 + 1.5 \times \text{IQR}$$

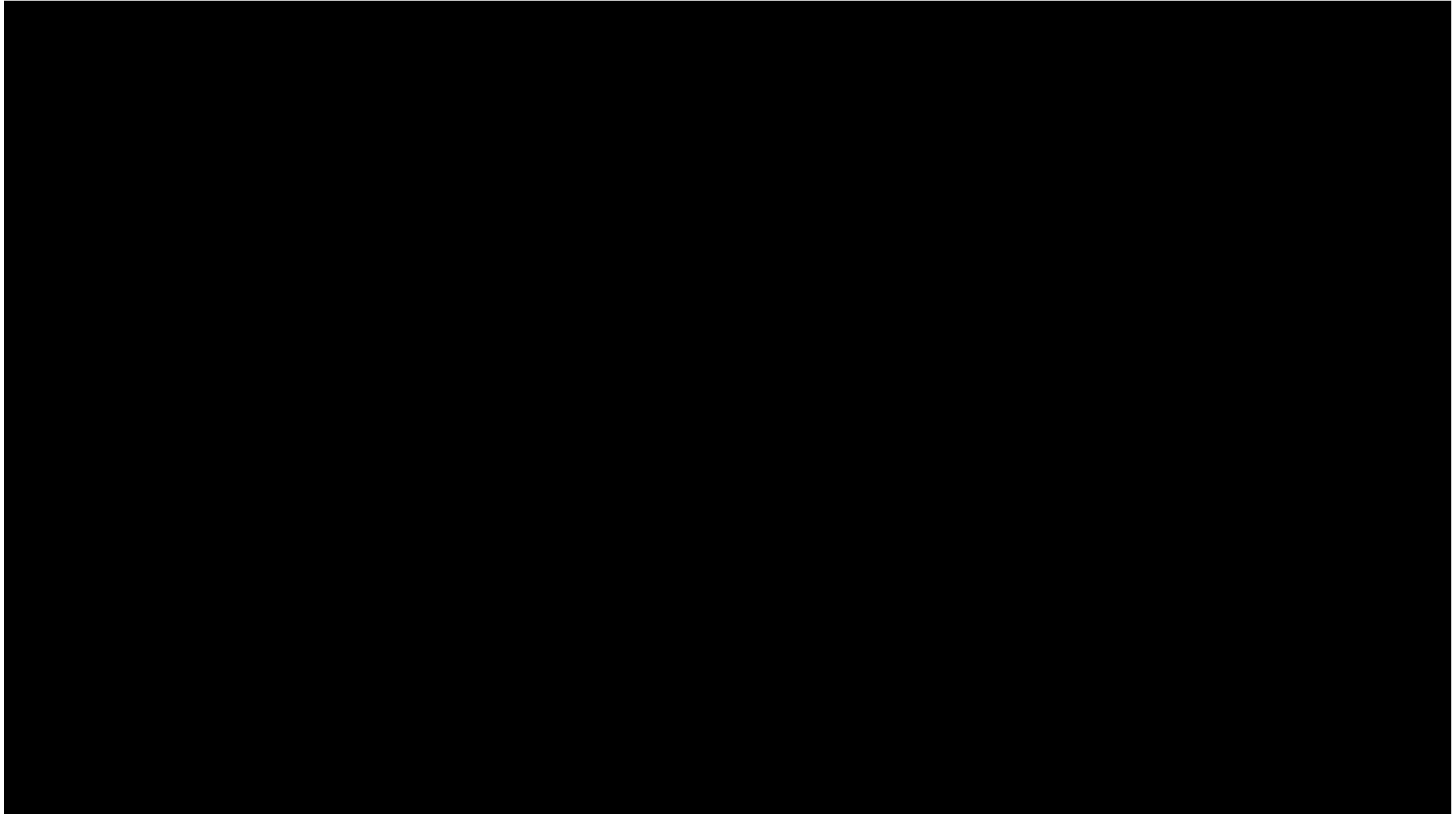
$$\text{Lower Bound} = Q1 - 1.5 \times \text{IQR}$$



$$\begin{aligned} \text{Interquartile range, IQR} &= Q3 - Q1 \\ &= 70 - 56 = 14 \end{aligned}$$

$$\begin{aligned} \text{Smallest allowed calculated non-outlier value} \\ &= Q1 - 1.5 \times \text{IQR} \\ &= 56 - 1.5 \times 14 = 35 \end{aligned}$$

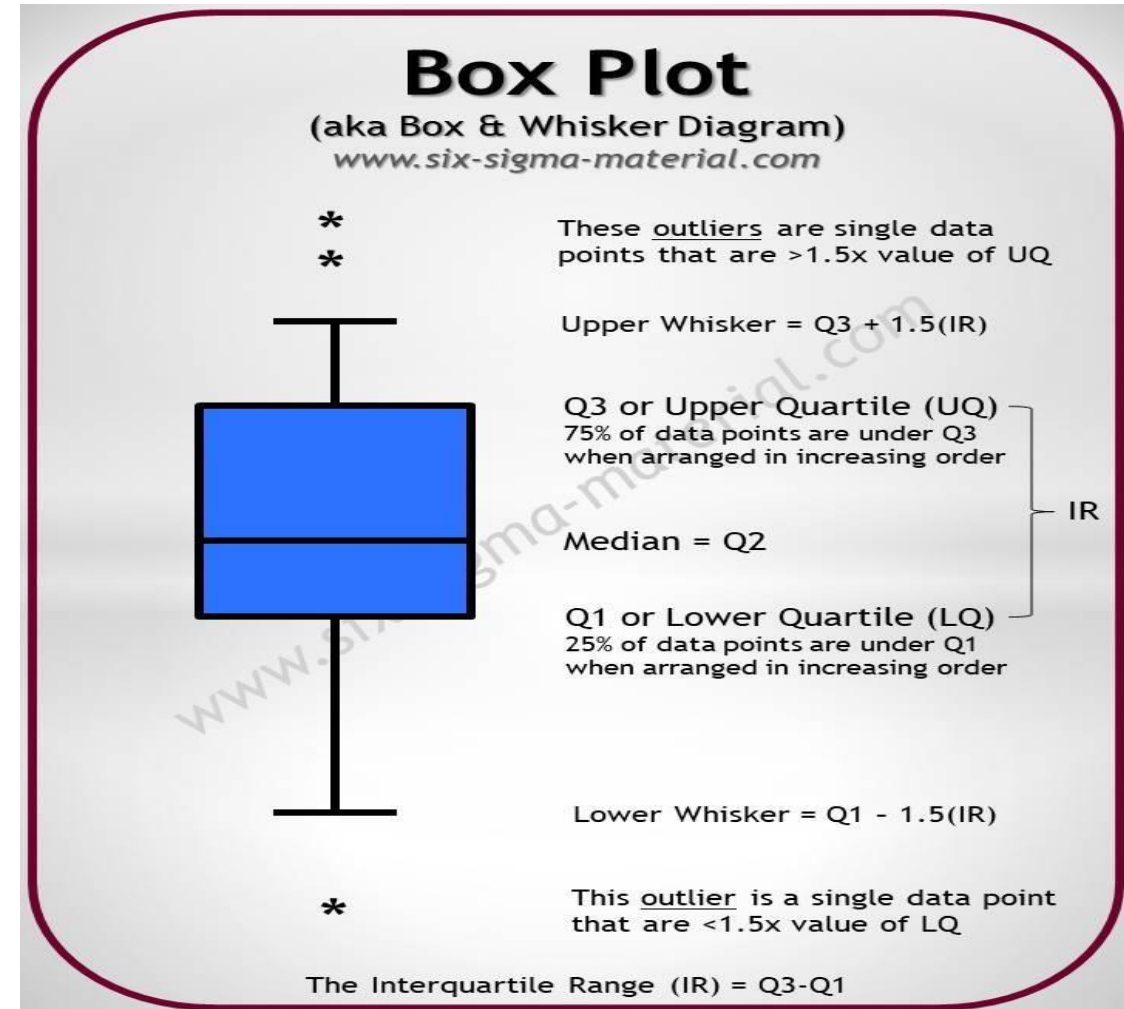
$$\begin{aligned} \text{Largest allowed calculated non-outlier value} \\ &= Q3 + 1.5 \times \text{IQR} \\ &= 70 + 1.5 \times 14 = 91 \end{aligned}$$



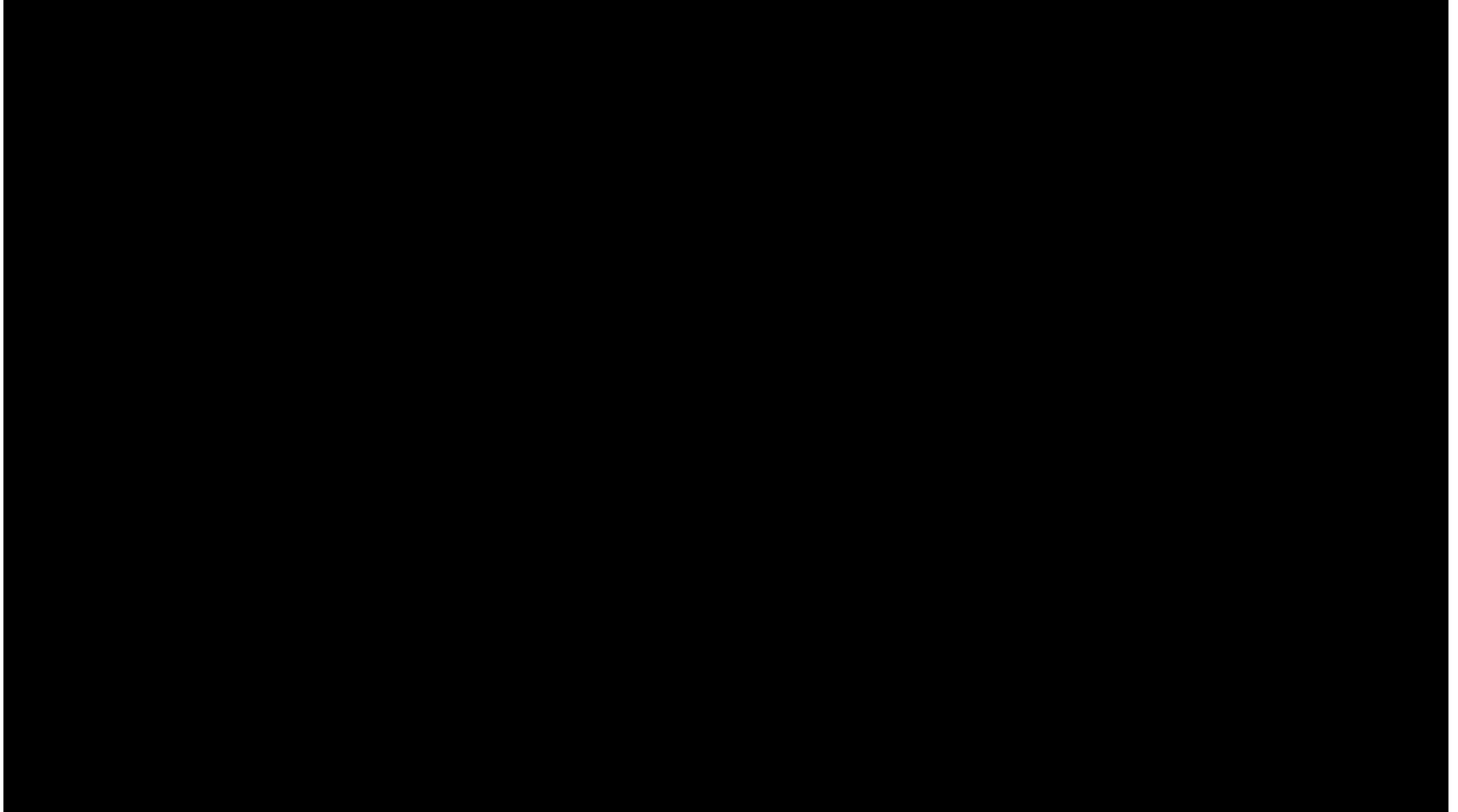
EDA Step 4: Data Visualization

Visual Inspection

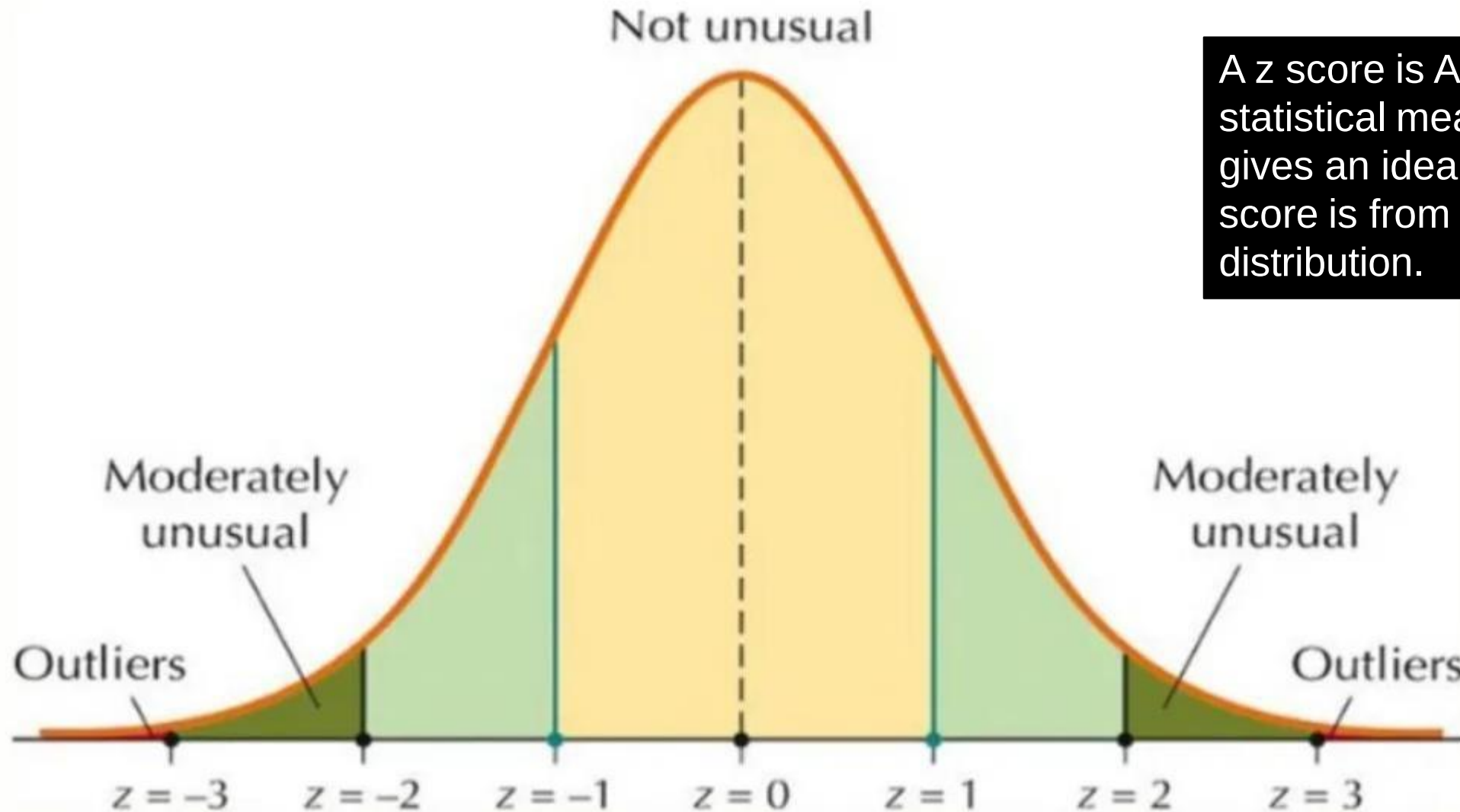
- Box Plot
 - Box plots provide a visual distribution summary, including the median, quartiles, and range.
 - Outliers can be spotted as individual points outside the whiskers.



How To Make Box and Whisker Plots



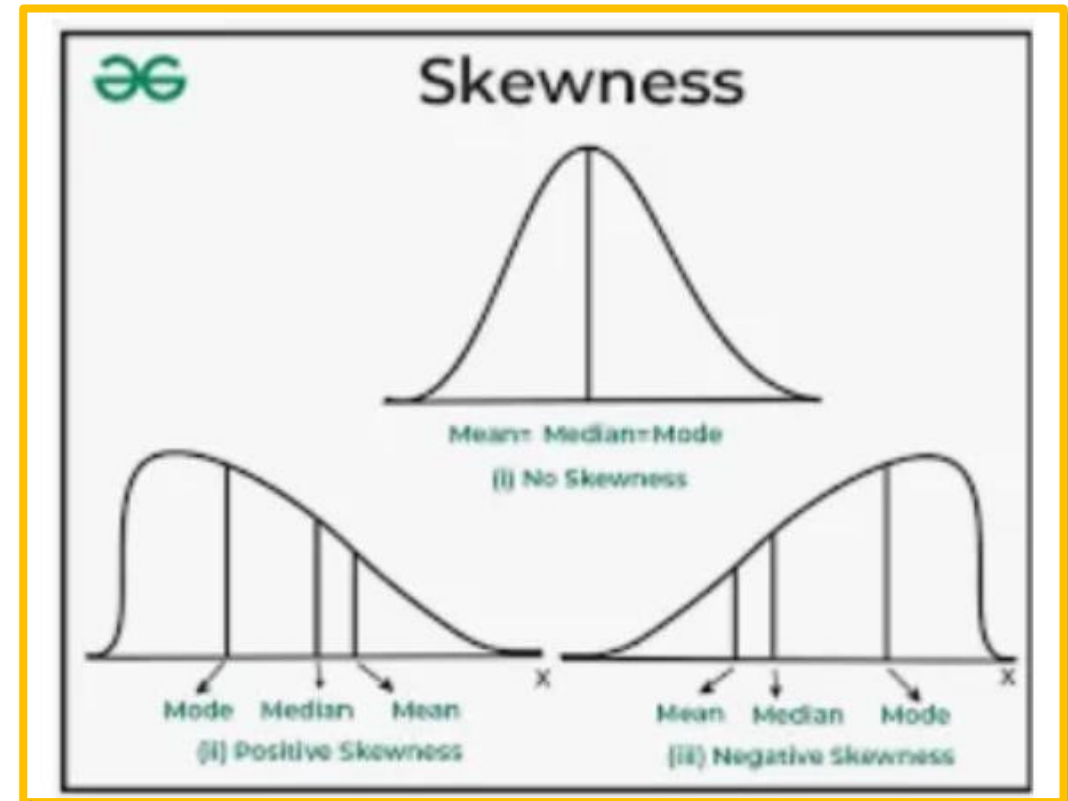
Detecting Outliers with z-Scores



A z score is ANOTHER statistical measurement that gives an idea of how far a raw score is from the mean of a distribution.

For example, a z-score of 2.5 indicates that the data point is 2.5 standard deviation away from the mean. Usually z-score =3 is considered as a cut-off value to set the limit. Therefore, any z-score greater than +3 or less than -3 is considered as outlier which is pretty much similar to standard deviation method.

*Z-Score only works with the data which is completely or close to **normally distributed**, which in turn stimulates that this method is not for **skewed** data, either **left skew** or **right skew**.*

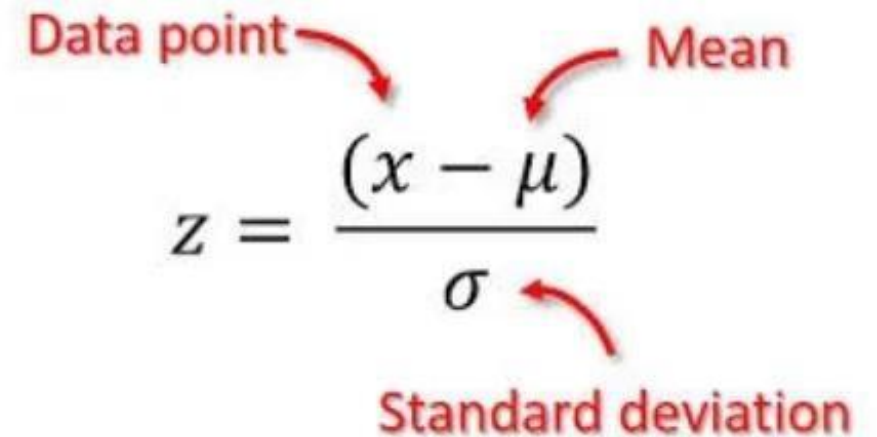


Problem 1:

For a psychological well-being scale ranging from 1 to 10 with an average score of 6 and a standard deviation of 2, calculate the z-score for a person who scored 4.

Problem 2:

In a study on sleep, participants report an average of 7 hours of sleep per night, with a standard deviation of 1 hour. Calculate the z-score for a person reporting 5 hours of sleep.



The diagram shows the z-score formula
$$z = \frac{(x - \mu)}{\sigma}$$
 with red arrows pointing to the components: 'Data point' points to x , 'Mean' points to μ , and 'Standard deviation' points to σ .

Problem 3:

In a happiness scale where the average score is 75 and the standard deviation is 10, find the z-score corresponding to a score of 95.

Problem 1:

For a psychological well-being scale ranging from 1 to 10 with an average score of 6 and a standard deviation of 2, calculate the z-score for a person who scored 4.

$$\text{Z-score} = (4 - 6) / 2 = -1.$$

Problem 2:

In a study on sleep, participants report an average of 7 hours of sleep per night, with a standard deviation of 1 hour.

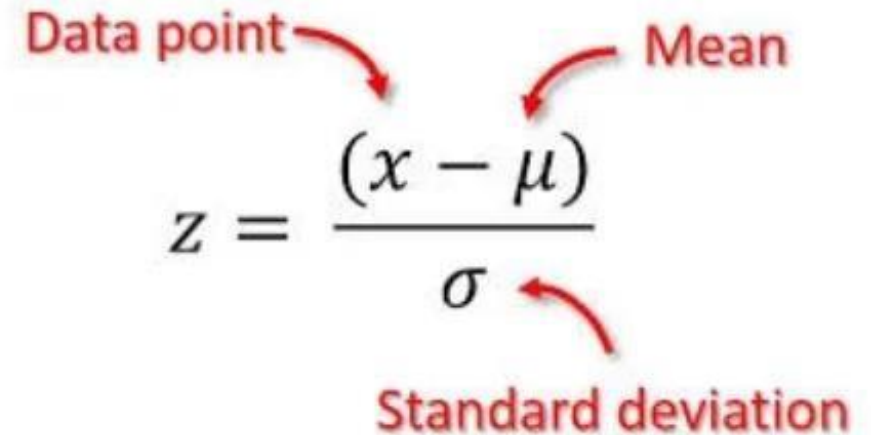
Calculate the z-score for a person reporting 5 hours of sleep.

$$\text{Z-score} = (5 - 7) / 1 = -2.$$

Problem 3:

In a happiness scale where the average score is 75 and the standard deviation is 10, find the z-score corresponding to a score of 95.

$$\text{Z-score} = (95 - 75) / 10 = 2.$$



$$Z = \frac{(x - \mu)}{\sigma}$$

The diagram shows the formula for the z-score with red arrows pointing to the components: 'Data point' points to x , 'Mean' points to μ , and 'Standard deviation' points to σ .

EDA

Leads to

Data Pre-Processing

Data Cleaning

- Handling Missing Value
- Handling Outliers

Data Transformation

- Feature Scaling
- Feature Encoding
- Feature Engineering

Handling Imbalanced Data

- Under Sampling
- Over Sampling

Data Reduction

- Dimensionality Reduction
- Instance Reduction

Data Splitting

Data Pre-Processing

Data Cleaning

- Handling Missing Value
- Handling Outliers

Data Transformation

- Feature Scaling
- Feature Encoding
- Feature Engineering

Handling Imbalanced Data

- Under Sampling
- Over Sampling

Data Reduction

- Dimensionality Reduction
- Instance Reduction

Data Splitting

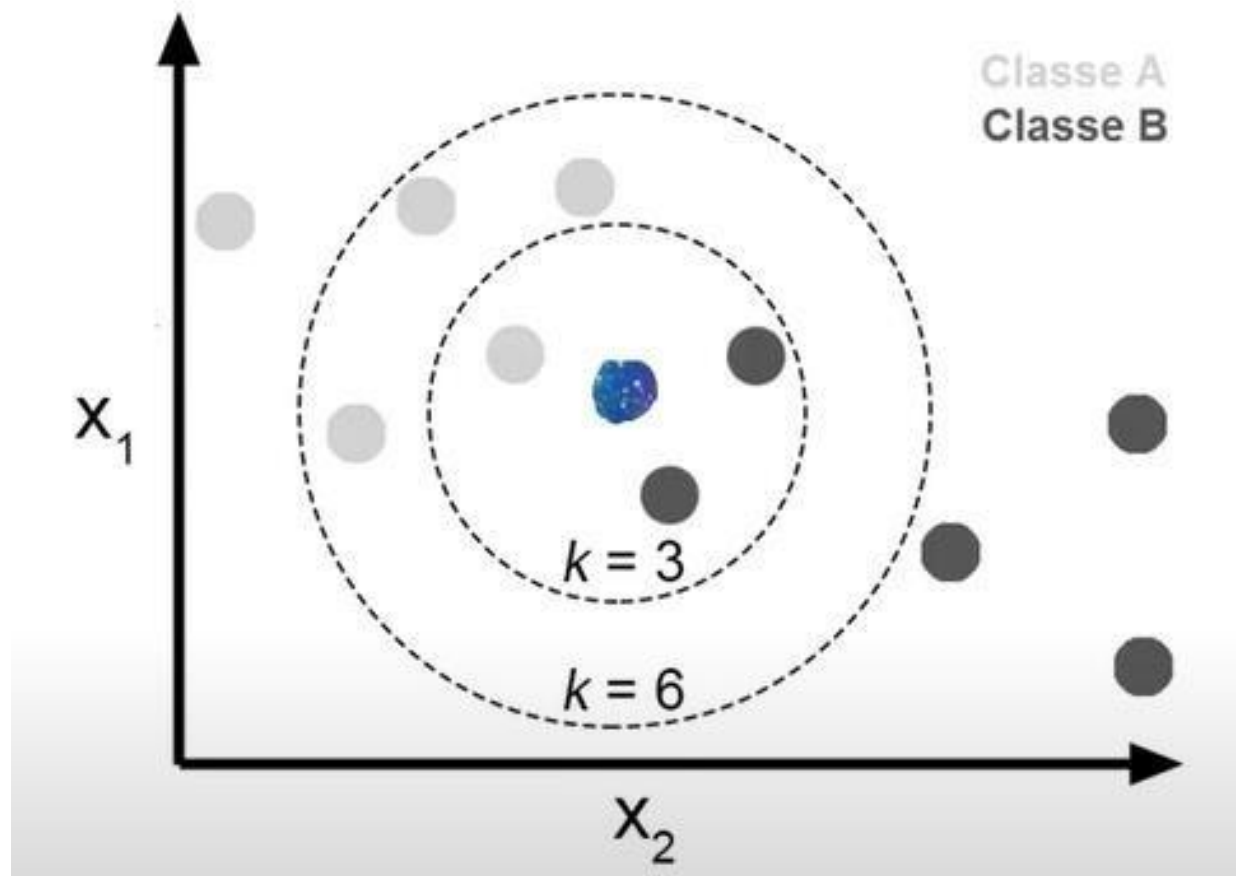
Data Pre-Processing: Data Cleaning

Data in the Real World Is Dirty: Lots of potentially incorrect data, e.g., instrument faulty, human or computer error, transmission error

- incomplete: lacking attribute values, lacking certain attributes of interest, or containing only aggregate data. e.g., *Occupation*=" " (missing data)
- Noisy/Outliers: containing noise, errors, or outliers.e.g., *Salary*="−10" (an error)
- inconsistent: containing discrepancies in codes or names, e.g.,*Age*="42", *Birthday*="03/07/2010", Was rating "1, 2, 3", now rating "A, B, C"
- Intentional (e.g., *disguised missing data*)
 - Jan. 1 as everyone's birthday?

Data Cleaning – Handling Missing Data

- **Removal of Missing Values:** Remove entire rows with missing values. This is suitable when the missing values are sporadic and removing a few rows doesn't significantly impact the dataset.
- **Imputation Techniques:**
 - **Mean, Median, or Mode Imputation:** Fill missing values with the mean, median, or mode of the non-missing values in the variable. This method is suitable for numerical data.
 - **Forward Fill (or Backward Fill):** Propagate the last observed value forward (or the next observed value backward) to fill in missing values in time-series or sequential data.
 - **K-Nearest Neighbors (KNN) Imputation:** Estimate missing values based on the values of their k-nearest neighbors. This method considers the similarity between data points.
- **Predictive Modelling:**
 - **Use Machine Learning Models:** Train a machine learning model on the data with missing values as the target variable. Predict the missing values using the trained model.



Identification of information- dropNotAvailiable function from Pandas

`pandas.DataFrame.info()`

```
>>> df.dropna(how='all')
   name      toy      born
0  Alfred    NaN     NaT
1  Batman  Batmobile 1940-04-25
2  Catwoman  Bullwhip    NaT
```

Keep only the rows with at least 2 non-NA values.

```
>>> df.dropna(thresh=2)
   name      toy      born
1  Batman  Batmobile 1940-04-25
2  Catwoman  Bullwhip    NaT
```

Define in which columns to look for missing values.

```
>>> df.dropna(subset=['name', 'toy'])
   name      toy      born
1  Batman  Batmobile 1940-04-25
2  Catwoman  Bullwhip    NaT
```

```
import numpy as np
import pandas as pd
import sklearn as sk
```

```
myDF=pd.DataFrame([[1,2,3],["Alice","Bob","Tom"],[52,80]])
```

```
myDF
```

	0	1	2
0	1	2	3
1	Alice	Bob	Tom
2	52	80	None

```
myDF.dropna()
```

	0	1	2
0	1	2	3
1	Alice	Bob	Tom

```
myDF.dropna(axis='columns')
```

	0	1
0	1	2
1	Alice	Bob
2	52	80

Removal of Missing Values - dropNotAvailiable function from Pandas

pandas.DataFrame.dropna()

```
>>> df.dropna(how='all')
   name      toy      born
0  Alfred    NaN      NaT
1  Batman  Batmobile  1940-04-25
2  Catwoman  Bullwhip      NaT
```

Keep only the rows with at least 2 non-NA values.

```
>>> df.dropna(thresh=2)
   name      toy      born
1  Batman  Batmobile  1940-04-25
2  Catwoman  Bullwhip      NaT
```

Define in which columns to look for missing values.

```
>>> df.dropna(subset=['name', 'toy'])
   name      toy      born
1  Batman  Batmobile  1940-04-25
2  Catwoman  Bullwhip      NaT
```

```
import numpy as np
import pandas as pd
import sklearn as sk
```

```
myDF=pd.DataFrame([[1,2,3],["Alice","Bob","Tom"],[52,80]])
```

myDF

	0	1	2
0	1	2	3
1	Alice	Bob	Tom
2	52	80	None

myDF.dropna()

	0	1	2
0	1	2	3
1	Alice	Bob	Tom

myDF.dropna(axis='columns')

	0	1
0	1	2
1	Alice	Bob
2	52	80

Identification of null values- `pandas.DataFrame.isnull()`

```
data.isnull().sum()
```

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

`SimpleImputer()` - SimpleImputer is a scikit-learn class which is helpful in handling the missing data in the predictive model dataset. It replaces the NaN values with a specified placeholder.

```
from sklearn.impute import SimpleImputer
```

```
numeric_features = data.select_dtypes(include=np.number)

imputer = SimpleImputer(strategy='mean')
imputed_data = pd.DataFrame(imputer.fit_transform(numeric_features),
                             columns=numeric_features.columns)

# Combine the imputed numeric features with the original non-numeric features
data = pd.concat([imputed_data, data[['species']]], axis=1)

data.isnull().sum() # Verify no missing values remain in numeric features
```

ACTIVITY

```
california = fetch_california_housing()
data = pd.DataFrame(california.data, columns=california.feature_names)
data['PRICE'] = california.target
```

?????

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	PRICE
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

?????

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	PRICE
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704	2.068558
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532	1.153956
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000	0.149990
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-121.800000	1.196000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000	1.797000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000	2.647250
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.000010

Display dataset information

?????

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	PRICE	20640 non-null	float64

```
dtypes: float64(9)
```

```
memory usage: 1.4 MB
```

```
None
```

ACTIVITY

```
california = fetch_california_housing()
data = pd.DataFrame(california.data, columns=california.feature_names)
data['PRICE'] = california.target
data.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	PRICE
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

data.describe()

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	PRICE
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704	2.068558
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532	1.153956
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000	0.149990
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-121.800000	1.196000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000	1.797000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000	2.647250
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.000010

```
# Display dataset information
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	PRICE	20640 non-null	float64

```
dtypes: float64(9)
memory usage: 1.4 MB
None
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
data.isnull().sum()
```

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

```
numeric_features = data.select_dtypes(include=np.number)
```

```
imputer = SimpleImputer(strategy='mean')
```

```
imputed_data = pd.DataFrame(imputer.fit_transform(numeric_features),
                             columns=numeric_features.columns)
```

```
# Combine the imputed numeric features with the original non-numeric features
data = pd.concat([imputed_data, data[['species']]], axis=1)
```

```
data.isnull().sum() # Verify no missing values remain in numeric features
```

```
data.isnull().sum()
```

```
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
PRICE       0
dtype: int64
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	PRICE
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

```
imputer = SimpleImputer(strategy='mean')
data = pd.DataFrame(imputer.fit_transform(data), columns=data.columns)
data.isnull().sum() # Verify no missing values remain
```

```
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
PRICE       0
dtype: int64
```

Filling of Missing Values - fillNotAvaliable function from Pandas

`pandas.DataFrame.fillna()`

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
2	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
3	R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
4	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
5	Amir Johnson	Boston Celtics	90.0	PF	29.0	6-9	240.0	NaN	12000000.0
6	Jordan Mickey	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0
7	Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0
8	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
newdf = df.fillna(22222)
```

```
# replacing na values in college with No college
nba["College"].fillna("No College", inplace = True)
```

```
nba["College"].fillna( method ='ffill', inplace = True)
```

```
df['A'].fillna(df['A'].median(), inplace=True)
```

Data Importing

1. `pd.read_csv()`
2. `pd.read_excel()`
3. `pd.read_sql()`
4. `pd.read_json()`
5. `pd.read_html()`
6. `pd.read_parquet()`
7. `pd.read_table()`
8. `pd.DataFrame()`
9. `pd.Series()`
10. `pd.concat()`

Data Cleaning

1. `df.dropna()`
2. `df.fillna()`
3. `df.drop_duplicates()`
4. `df.replace()`
5. `df.isnull()`
6. `df.notnull()`
7. `df.astype()`
8. `df.sort_values()`
9. `df.apply()`
10. `df.groupby()`

Data Statistics

1. `df.head()`
2. `df.describe()`
3. `df.mean()`
4. `df.median()`
5. `df.mode()`
6. `df.sum()`
7. `df.std()`
8. `df.var()`
9. `df.corr()`
10. `df.cov()`

Data Pre-Processing

Data Cleaning

- Handling Missing Value
- Handling Outliers

Data Transformation

- Feature Scaling
- Feature Encoding
- Feature Engineering

Handling Imbalanced Data

- Under Sampling
- Over Sampling

Data Reduction

- Dimensionality Reduction
- Instance Reduction

Data Splitting

Data Transformation

- Feature Scaling(For Numerical Data)

- **Feature Scaling:** Scale the numerical features to a common scale to avoid issues caused by different ranges or units.
- Scaling the features can help the model to converge faster, and can also help to *reduce the impact of outliers in the data*.
- Common scaling methods include:
 - **Standardization / Z-Score Normalization:** $X_{\text{new}} = (X - \text{mean}) / \text{Std}$. It is not bound to a certain range.
 - **Normalization / min-max scaling:** $X_{\text{new}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$. This scales the range to [0, 1]



Feature Scaling Methods

Absolute Maximum Scaling

Formula:

$$x' = \frac{x}{\max(|x|)}$$

Use `MaxAbsScaler` from `sklearn.preprocessing`.

Min-Max Scaling

Formula:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

In Python, use `MinMaxScaler` from `sklearn.preprocessing`.

Robust Scaling

Formula:

$$x' = \frac{x - Q1}{Q3 - Q1}$$

Use `RobustScaler` from `sklearn.preprocessing`.

Standardization

Formula:

$$z = \frac{x - \mu}{\sigma}$$

Use `StandardScaler` from `sklearn.preprocessing`.

Data(v)
1000
2000
3000
5000
9000

Min-Max Normalization

$$V = \frac{x - \min}{\max - \min} \quad \min = 1000 \text{ and } \max = 9000$$

$$V = \frac{1000 - 1000}{9000 - 1000} = 0$$

$$V = \frac{2000 - 1000}{9000 - 1000} = 0.125$$

$$V = \frac{3000 - 1000}{9000 - 1000} = 0.25$$

$$V = \frac{5000 - 1000}{9000 - 1000} = 0.5$$

$$V = \frac{9000 - 1000}{9000 - 1000} = 1$$

Normalized Data(v)
0
0.125
0.25
0.5
1

$$\text{Standard Deviation} = \sqrt{\frac{\sum (x_i - \mu)^2}{n - 1}}$$

$$= \sqrt{\frac{(1000 - 4000)^2 + (2000 - 4000)^2 + (3000 - 4000)^2 + (5000 - 4000)^2 + (9000 - 4000)^2}{5 - 1}}$$

$$= 2489.97$$

$$\text{Mean} = \frac{(1000 + 2000 + 3000 + 5000 + 9000)}{5} = 4000$$

Z-Score Normalization

$$z = \frac{x - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation

Z-Score Normalization

$$Z = \frac{(x - \mu)}{\sigma}$$

$$V = \frac{1000 - 4000}{2489.97} = -1.204$$

$$V = \frac{2000 - 4000}{2489.97} = -0.803$$

$$V = \frac{3000 - 4000}{2489.97} = -0.4016$$

$$V = \frac{5000 - 4000}{2489.97} = 0.4016$$

$$V = \frac{9000 - 4000}{2489.97} = 2.008$$

Normalized Data(v)
-1.204
-0.803
-0.4016
0.4016
2.008

```
from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler()  
stock_df = scaler.fit_transform(stock_df)
```

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
stock_df = scaler.fit_transform(stock_df)
```

```
{  
CODE-Tute  
..  
}
```

Sl. No	Normalization	Standardization
1	Feature scaling method to bring the data into common range such as [0, 1], [-1, 1], etc.	Feature scaling method bring the data with mean 0 and unit variance
2	Scikit-learn provides MinMaxScaler, MaxAbsScaler and RobustScaler methods for normalization	Scikit-learn provides StandardScaler for standardization
3	MinMaxScaler and MaxAbsScaler are sensitive to outliers whereas RobustScaler is more robust to outliers	Standardization is less sensitive to outliers compared to MinMaxScaler and MaxAbsScaler
4	Useful when we don't know about the distribution of features and there are no or little outliers - MinMaxScaler: if features don't follow normal distribution and if there are no or less outliers - MaxAbsScaler: if the data is sparse - RobustScaler: if the data contains outliers	Useful when we know features are normally distributed (Gaussian distribution)

$$X_{\text{new}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

$$X_{\text{new}} = (X - \text{mean}) / \text{Std Deviation}$$

	S&P 500 PRICE (ORIGINAL)		S&P 500 PRICE (STANDARDIZED)
Randomly selected Value	2206	STANDARIZATION 	-0.583
Average	2319		0
Maximum	3000		3.513
Minimum	1800		-2.67

- S&P 500 PRICE (Mean) = 2319
- S&P 500 PRICE (Std) = 193.8

“A normalized dataset will always range from 0 to 1”

“A standardized dataset will always have a mean of 0 and standard deviation of 1, but can have any upper and lower values”



When

Normalization / min-max scaling

- when the values has a non-uniform distribution/skewness
- brings all the variables in the same scale within the range of 0 and 1
- should not be applied to categorical variables, only numerical variables.
- Apply when less outliers

When

Standardization / Z-Score Normalization

- when the variable has a uniform distribution
- scales a variable to have a mean of zero and a standard deviation of one
- should not be applied to categorical variables, only numerical variables.
- Apply when more outliers

- In case of neural networks, normalization is preferred since we don't assume any data distribution.
- Standardization is preferred when data follows gaussian distribution
- Standardization is preferred over normalization when there are a lot of outliers.

Test Set Classification Performance			
Algorithm	Without Feature Scaling	With Feature Scaling	Scaling Required?
Logistic Regression	0.53	0.70	YES
Support Vector Classifier	0.72	0.94	YES
MLP Classifier	0.73	0.89	YES
kNN Classifier	0.66	0.93	YES
Decision Tree	0.83	0.83	NO
Random Forest	0.91	0.91	NO
Gradient Boosting	0.86	0.86	NO
Naive Bayes	0.75	0.75	NO



Better performance with feature scaling

Same performance irrespective of feature scaling

Data Transformation

- Feature Encoding(For CatagoricalData)

- **Feature Encoding:** Convert categorical features into numerical representations that machine learning algorithms can understand.
- **Nominal Encoding:** there is no inherent order or ranking between the category data
 - **One-hot encoding:** Convert categorical variables into binary vectors, creating a binary column for each category.
 - **Label encoding:** Label Encoding is appropriate when encoding target variables, especially for categorical variables with no inherent order. E.g., Red:1, Blue:2, Yellow:3
- **Ordinal encoding:** Ordinal Encoding is suitable when categorical variables have an inherent order or ranking. In ordinal encoding, labels are translated to numbers based on their ordinal relationship to one another. E.g., Small:1, Medium:2, Large:3

1) One-hot encoding

Color
Green
Red
Black
Orange

----->

Green	Red	Black	Orange
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

4) Label encoding

Color
Green
Red
Black
Green

Assign unique labels to each category
----->

Encoding
1
2
3
1

5) Ordinal encoding

Size
XS
S
L
M

Assign unique labels to each category
----->

Encoding
1
2
4
3

ordered categories

6) Count encoding

Color
Green
Red
Black
Green

Encode based on frequency
----->

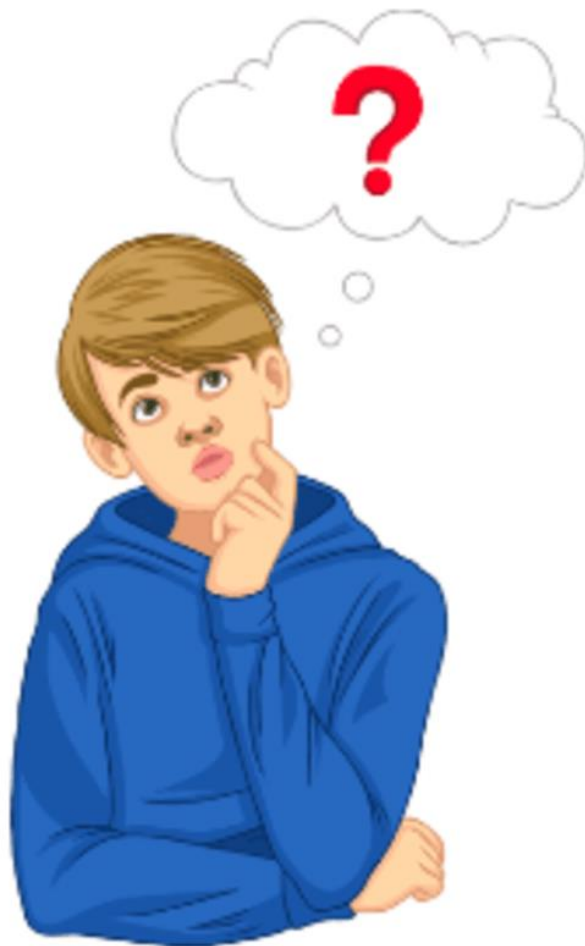
Encoding
2
1
1
2

7) Binary encoding

Color
Green
Red
Black
Green

Assign binary labels
----->

Color_0	Color_1
0	0
0	1
1	0
1	1



```
import pandas as pd
```

```
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Age': [20, 25, 20],
    'Salary': [3000, 4000, 3000]
})
```

```
df_encoded = pd.get_dummies(df, columns=['Age', 'Salary'])
print(df_encoded)
```

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Age': [20, 25, 20],
    'Salary': [3000, 4000, 3000]
})

df_encoded = pd.get_dummies(df, columns=['Age', 'Salary'])
print(df_encoded)
```

	Name	Age_20	Age_25	Salary_3000	Salary_4000
0	A	1	0	1	0
1	B	0	1	0	1
2	C	1	0	1	0

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Age': [20, 25, 20],
    'Salary': [3000, 4000, 3000]
})

# Create a label encoder object
le = LabelEncoder()

# Apply label encoding to the selected columns
for col in ['Age', 'Salary']:
    df[col] = le.fit_transform(df[col])

print(df)
```



```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
```

```
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Age': [20, 25, 20],
    'Salary': [3000, 4000, 3000]
})
```

Create a label encoder object

```
le = LabelEncoder()
```

Apply label encoding to the selected columns

```
for col in ['Age', 'Salary']:
    df[col] = le.fit_transform(df[col])
```

```
print(df)
```

	Name	Age	Salary
0	A	0	0
1	B	1	1
2	C	0	0



```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Sample DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D'],
    'Age': [20, 25, 30, 25],
    'Salary': [3000, 4000, 5000, 3000]
})

# Create encoder
encoder = OrdinalEncoder()

# Apply on specific columns
df[['Age', 'Salary']] = encoder.fit_transform(df[['Age', 'Salary']])

print(df)
```

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder
```

```
# Sample DataFrame
```

```
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D'],
    'Age': [20, 25, 30, 25],
    'Salary': [3000, 4000, 5000, 3000]
})
```

```
# Create encoder
```

```
encoder = OrdinalEncoder()
```

```
# Apply on specific columns
```

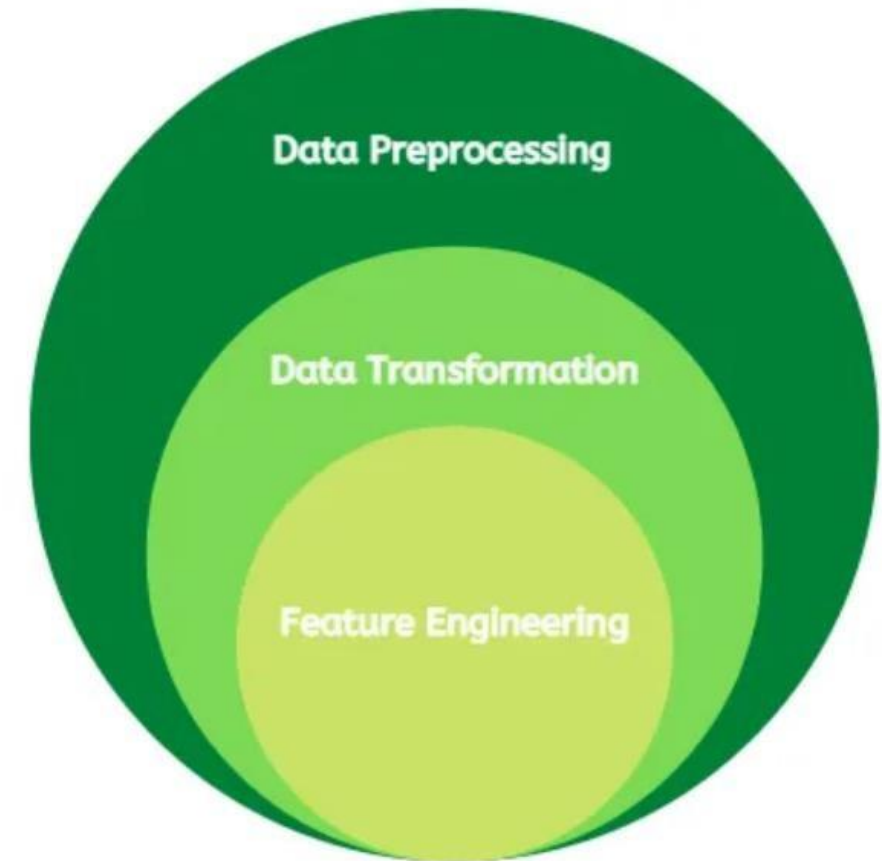
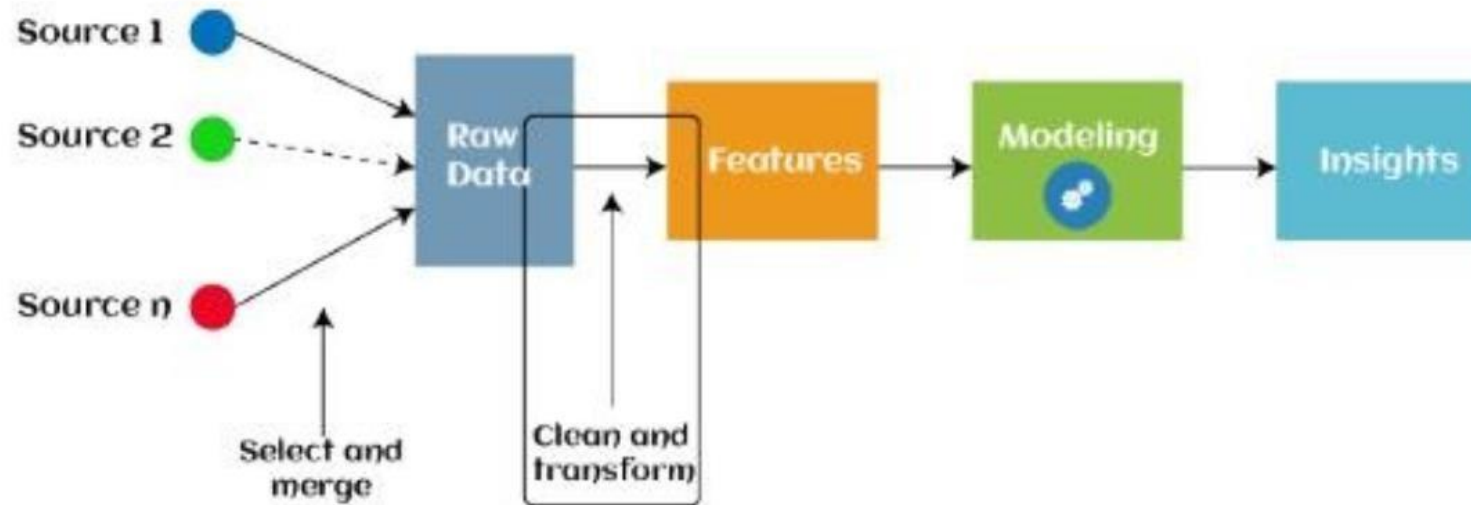
```
df[['Age', 'Salary']] = encoder.fit_transform(df[['Age', 'Salary']])
```

```
print(df)
```

	Name	Age	Salary
0	A	0.0	0.0
1	B	1.0	1.0
2	C	2.0	2.0
3	D	1.0	0.0

Data Transformation - Feature Engineering

Feature Engineering: Create new features or derive meaningful information from existing features to enhance the predictive power of the dataset.



Data Pre-Processing

Data Cleaning

- Handling Missing Value
- Handling Outliers

Data Transformation

- Feature Scaling
- Feature Encoding
- Feature Engineering

Handling Imbalanced Data

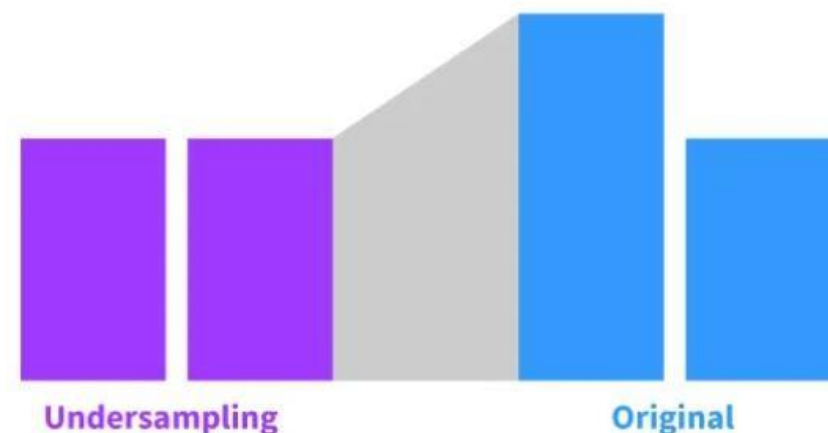
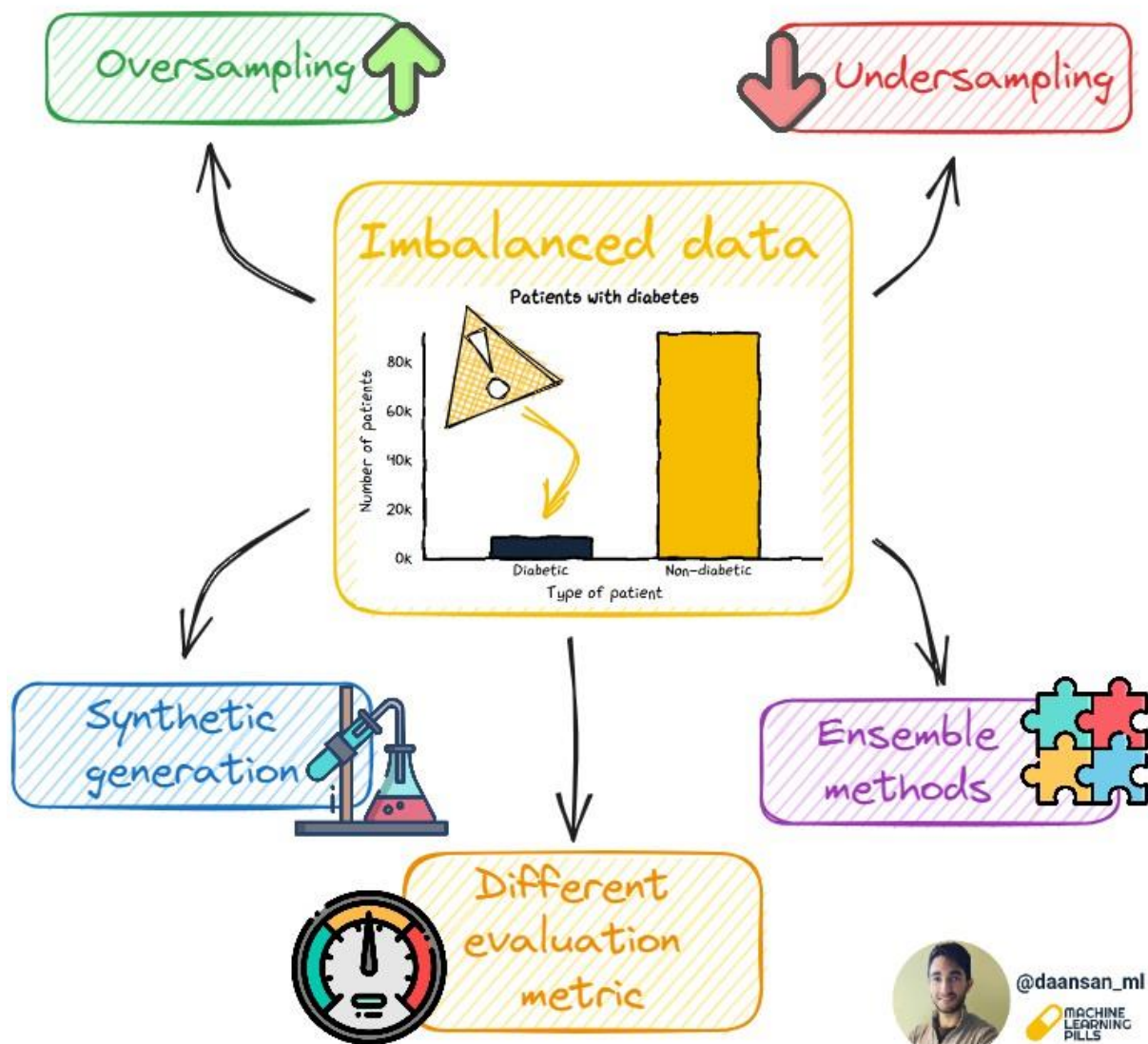
- Under Sampling
- Over Sampling

Data Reduction

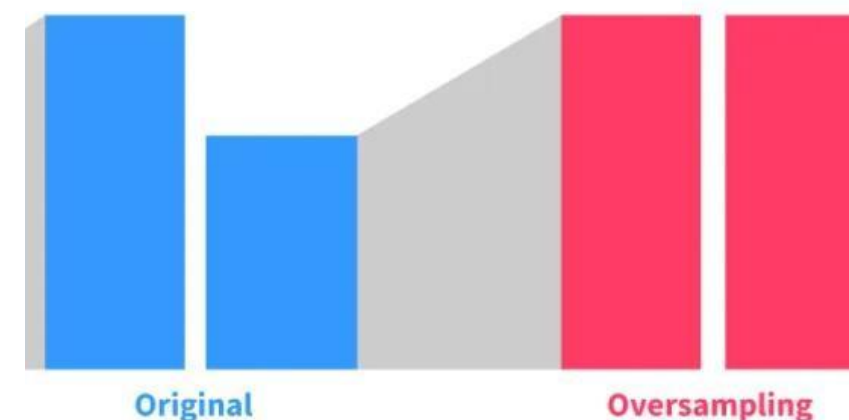
- Dimensionality Reduction
- Instance Reduction

Data Splitting

How to address imbalanced data?



In undersampling, we pull all the rare events while pulling a sample of the abundant events in order to equalize the datasets.



These methods can be used separately or together; one is not better than the other. Which method a data scientist uses depends on the dataset and analysis.



@daansan_ml



Data Pre-Processing – Handling Imbalanced Data

- Address class imbalance issues, where one class is significantly underrepresented compared to others.
- **Under Sampling:** Under-sampling is a technique used to balance the class distribution in a dataset by reducing the number of instances in the majority class.
 - **Random Under sampling:** Randomly remove instances from the majority class until a more balanced distribution is achieved.
 - **Cluster-Based Under-sampling:** Identify clusters in the majority class and retain only the centroids of these clusters.
 - **Tomek Links:** Identify pairs of instances (one from the majority class and one from the minority class) that are nearest neighbors but of different classes. Remove the majority class instance of each pair.
 - **Edited Nearest Neighbors:** Identify instances in the majority class that are misclassified by their nearest neighbors (based on a specified metric) and remove them.

Cluster-Based Under-sampling:

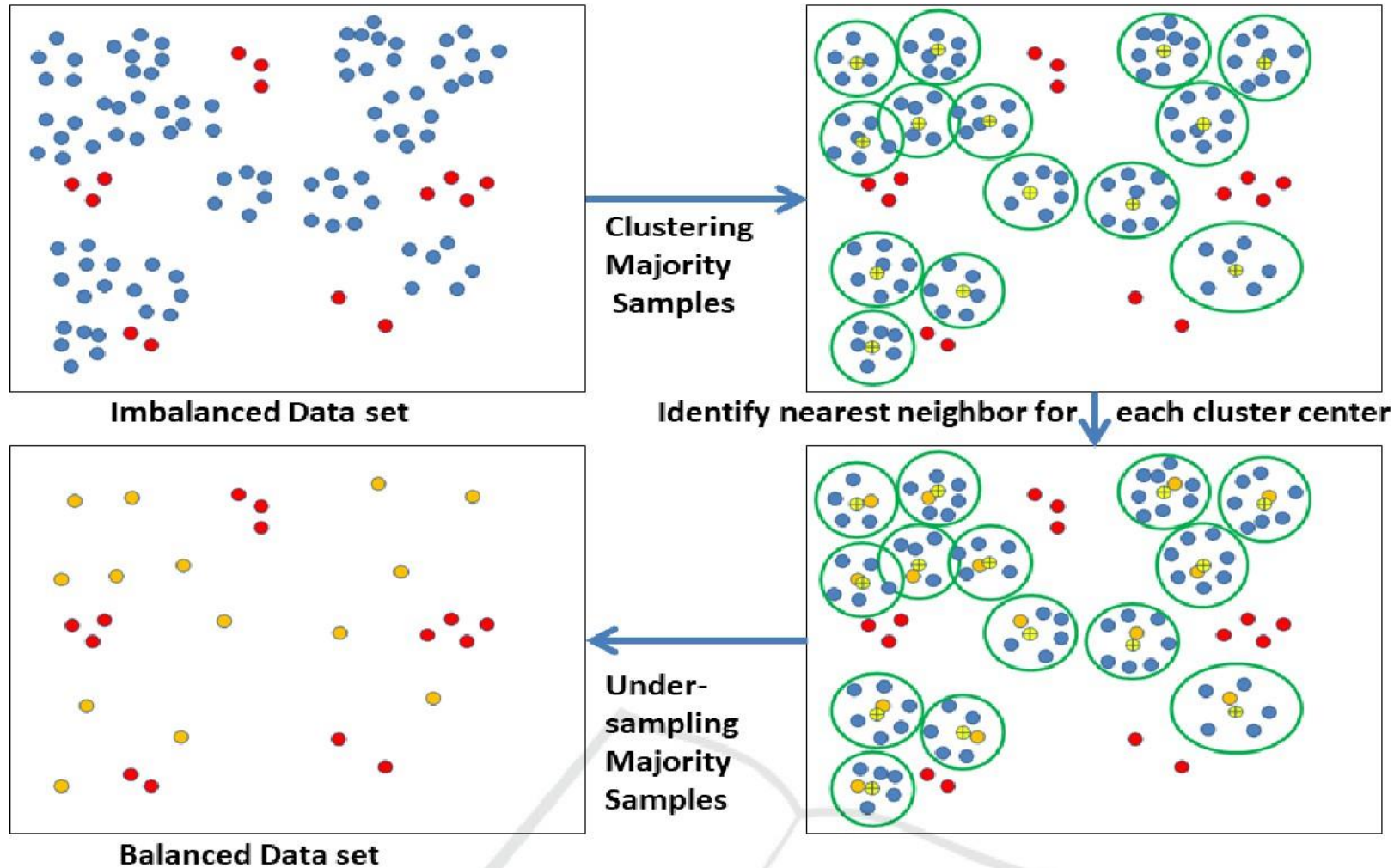
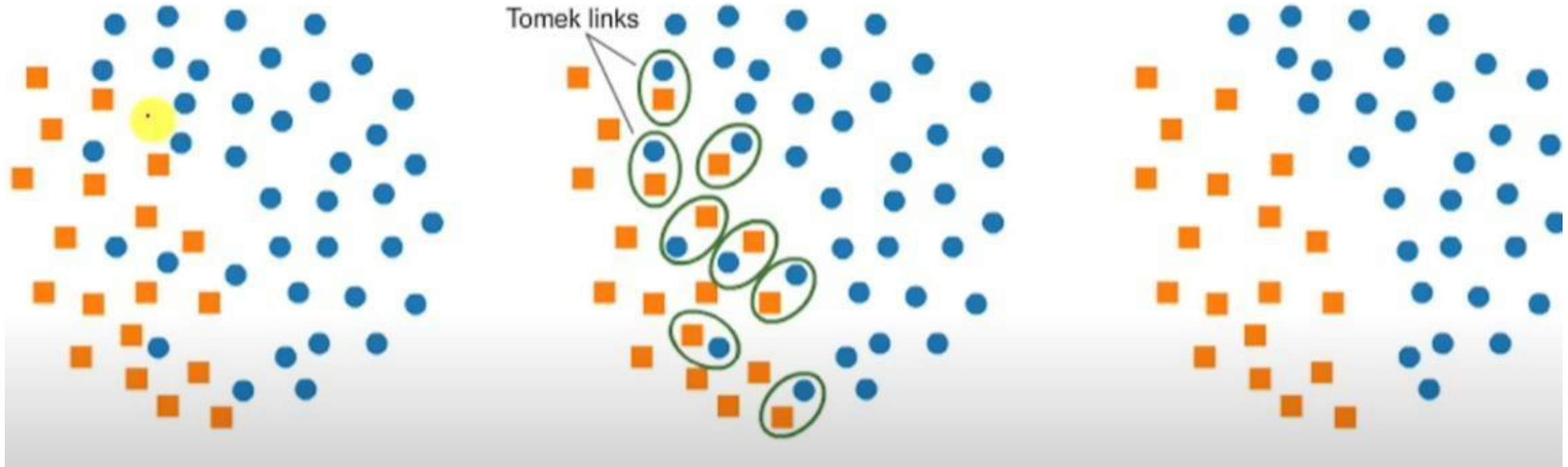


Figure 1: Clustering-based Under-sampling majority samples.

a technique that reduces the number of samples from the majority class in a dataset with an imbalanced class distribution.

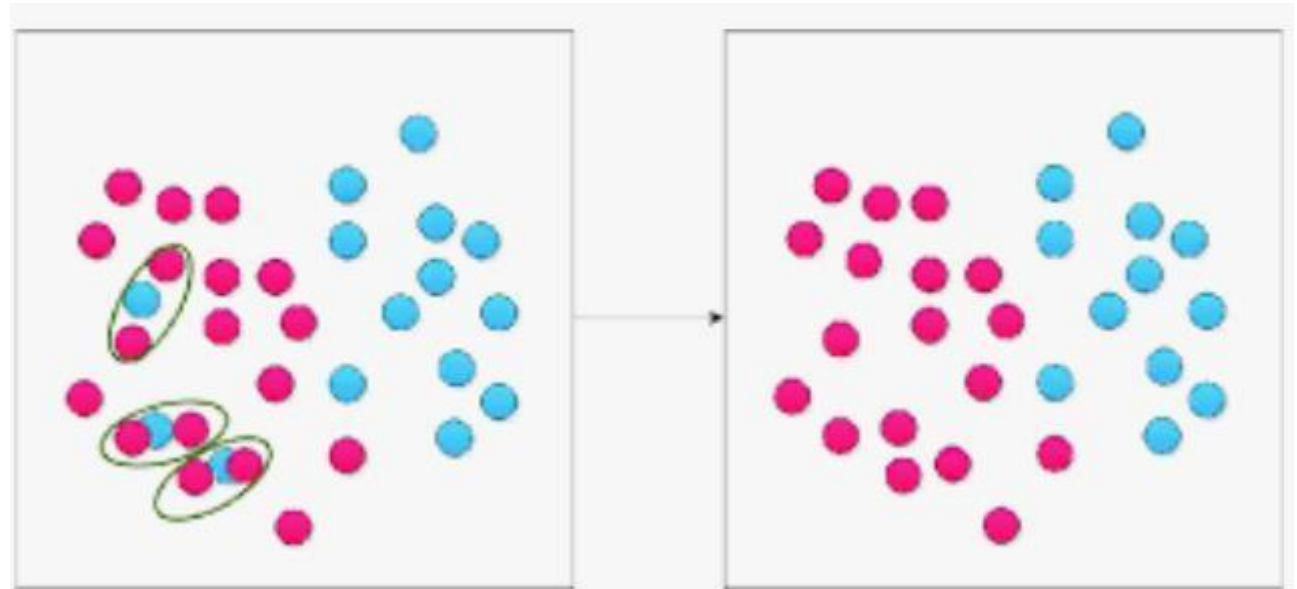
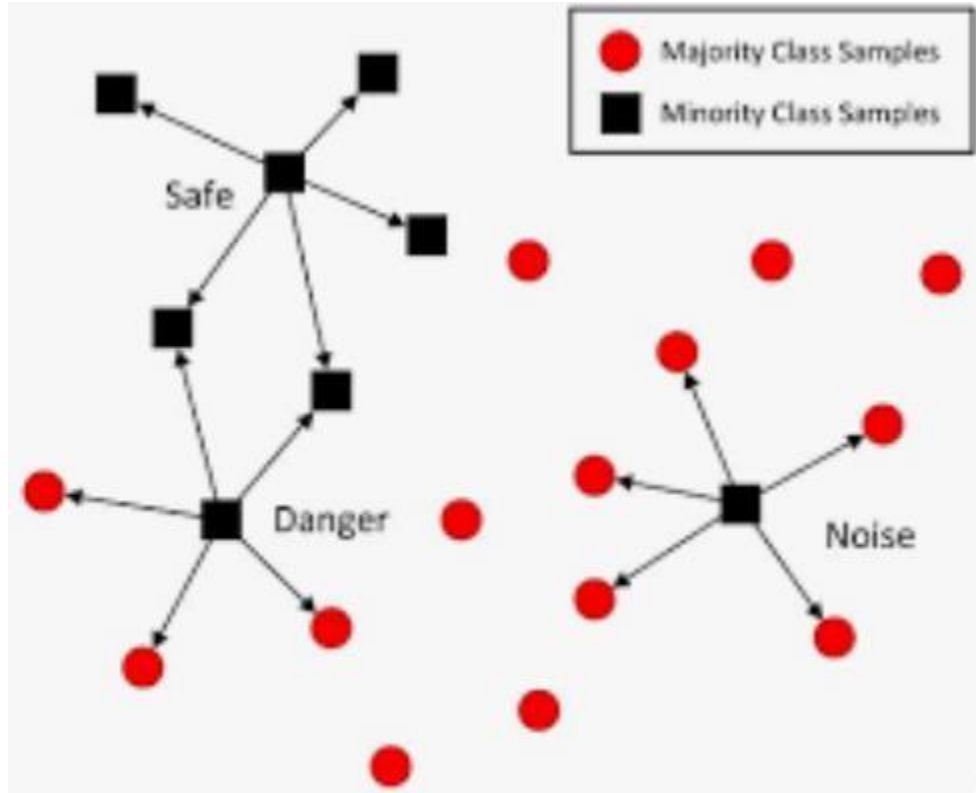
Tomek Link Under-sampling

a technique used to address class imbalance in machine learning datasets by identifying and removing data points from the majority class that are very close to data points from the minority class, essentially "cleaning up" the decision boundary between the classes by removing noisy or ambiguous data points considered "Tomek Links."



Edited Nearest Neighbors Under-sampling

a technique used to address class imbalance in datasets by selectively removing data points from the majority class that are considered "noisy" or misclassified based on their nearest neighbors

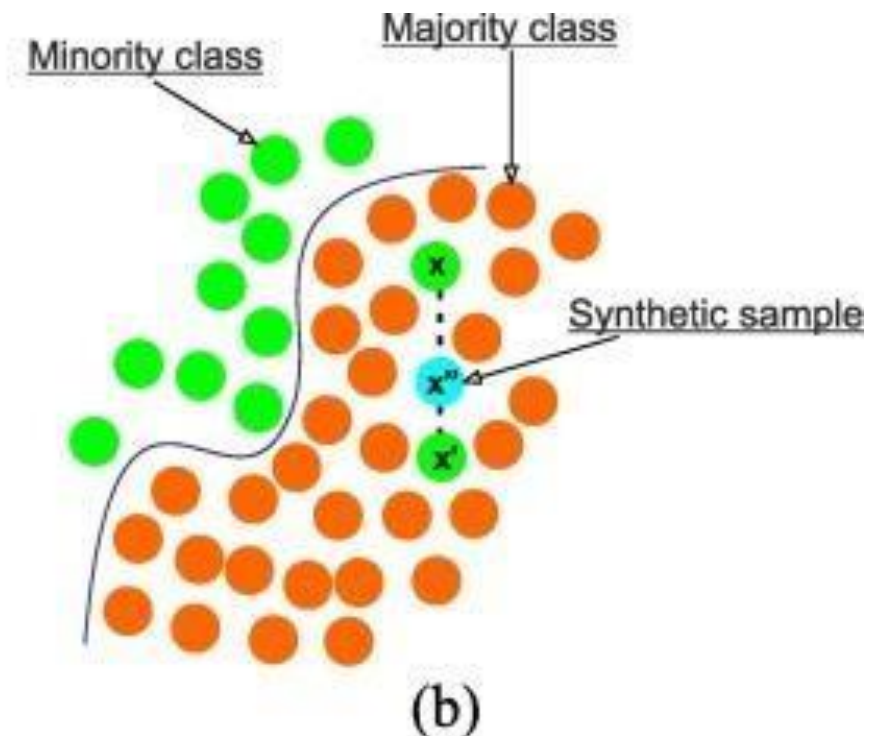
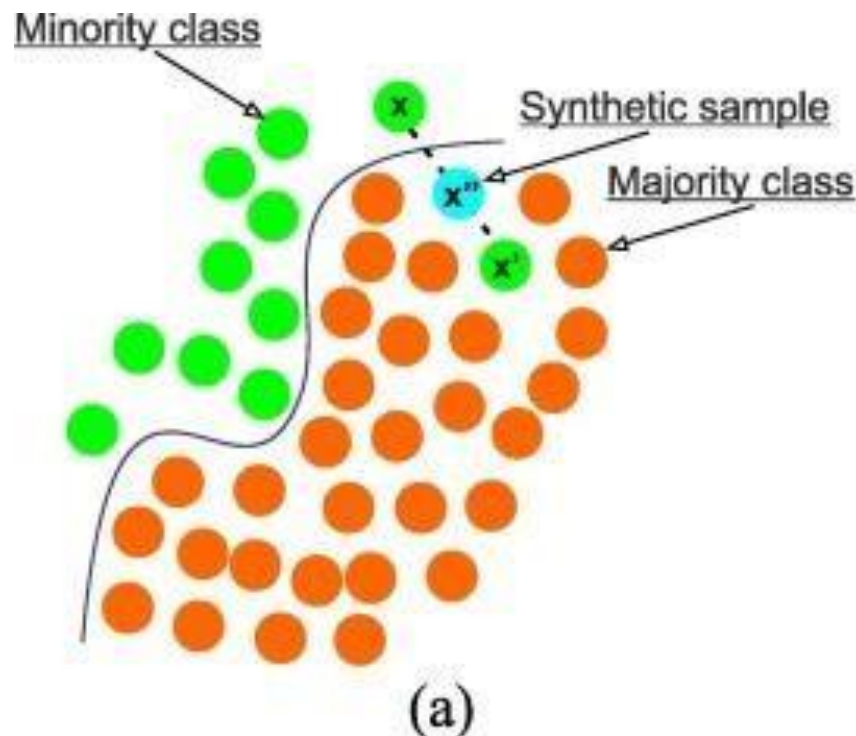


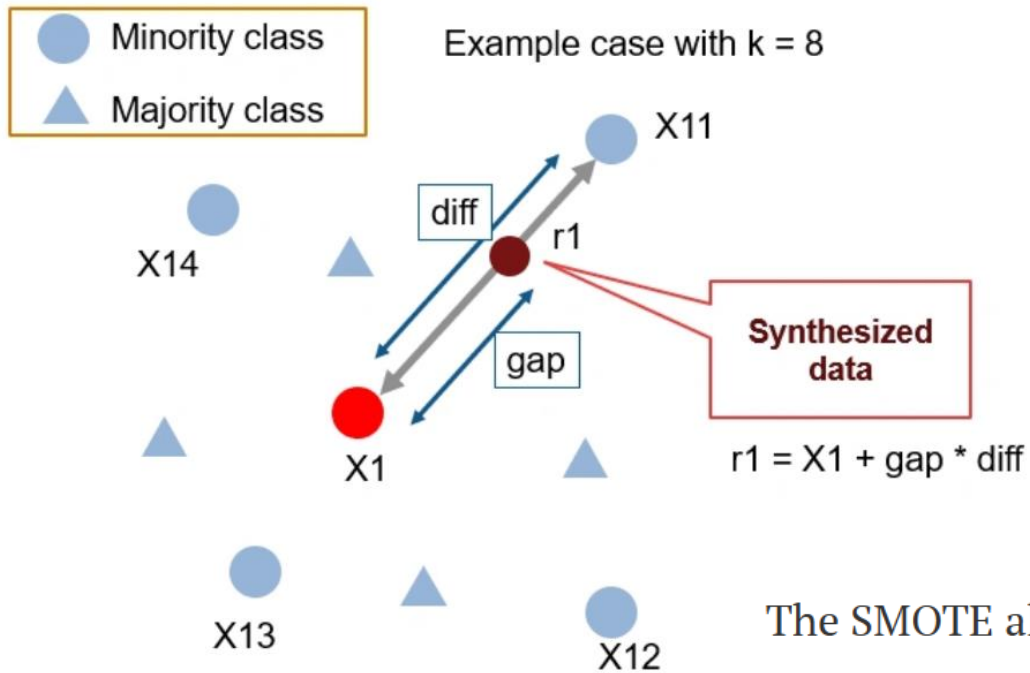
Data Pre-Processing – Handling Imbalanced Data

- **Over Sampling:** Oversampling is a technique used to balance the class distribution in a dataset by increasing the number of instances in the minority class.
 - **Random Oversampling:** Randomly duplicate instances from the minority class until a more balanced distribution is achieved.
 - **SMOTE (Synthetic Minority Over-sampling Technique):** Generates synthetic instances for the minority class by interpolating between existing instances.
 - **ADASYN (Adaptive Synthetic Sampling):** Similar to SMOTE but introduces adaptive weights to generate more synthetic examples for instances that are harder to learn.
 - **Borderline-SMOTE:** Applies SMOTE selectively to instances near the decision boundary (borderline instances).

SMOTE (Synthetic Minority Over-sampling Technique):

based on sampling data from the minority class by simply generating data points on the line segment connecting a randomly selected data point and one of its K-nearest neighbors.





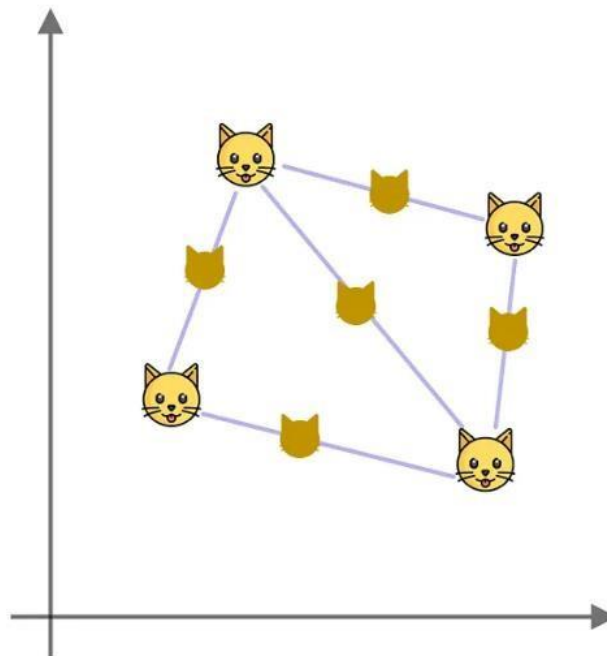
The SMOTE algorithm works in the following way:

1. Select a minority class sample from the original dataset.
2. Find its k nearest minority class neighbors in the feature space.
3. Randomly select one of the k nearest neighbors.
4. Generate a new synthetic sample by interpolating between the selected minority class sample and the randomly selected neighbor.
5. Repeat steps 1-4 until the desired number of synthetic samples is generated.

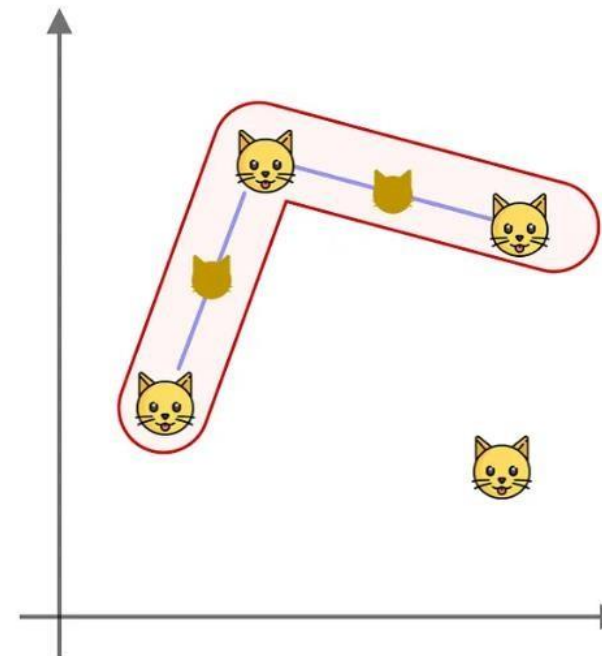
Borderline-SMOTE Over-sampling Technique :

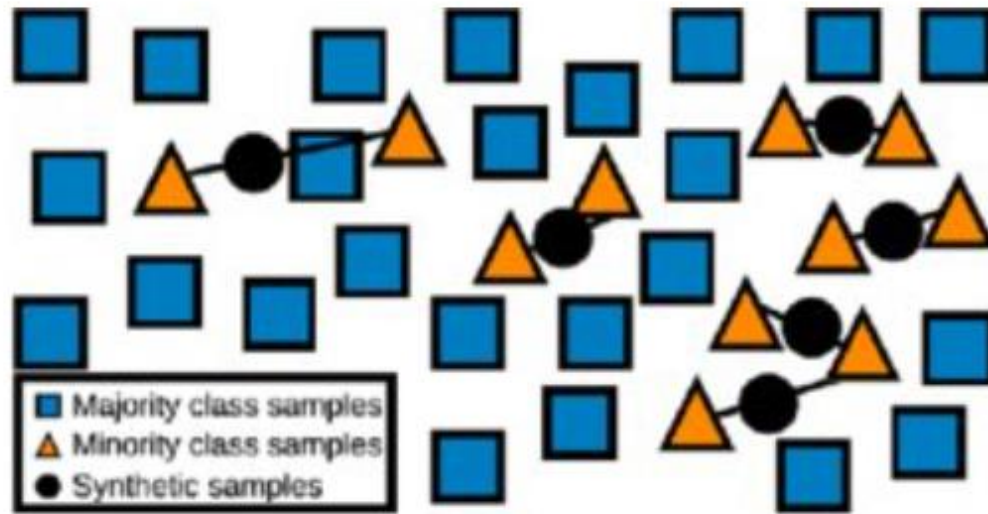
generates new examples of the minority class using the nearest neighbors of these cases in the border region between cases

SMOTE

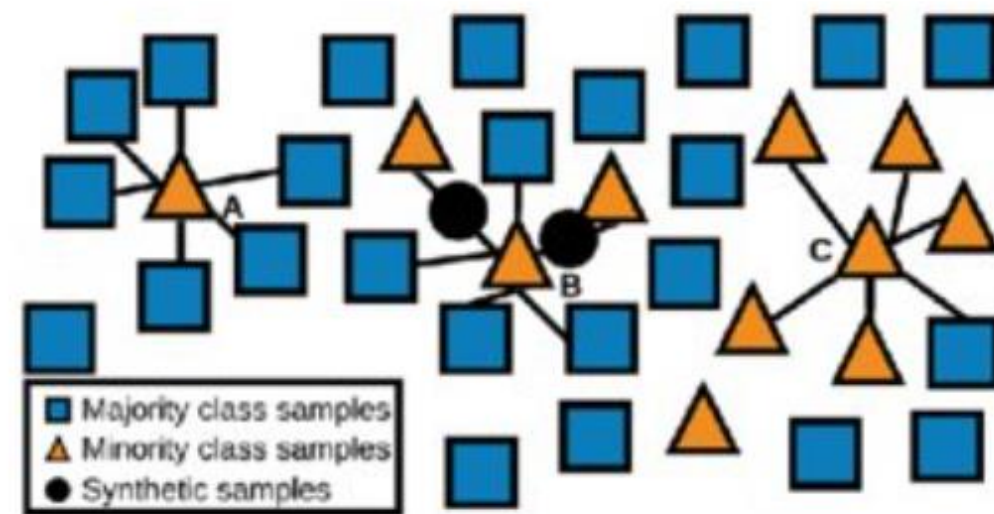


Borderline-SMOTE





(a) SMOTE example

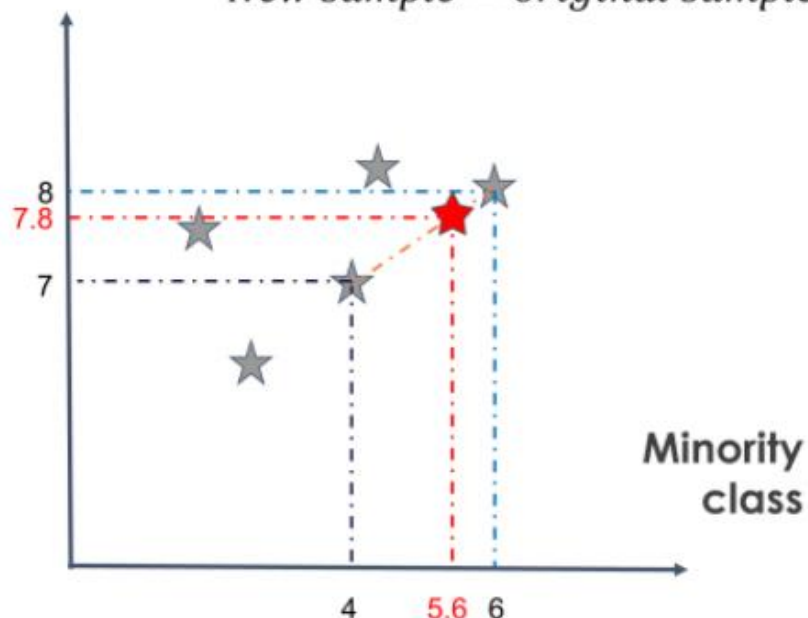


(b) BORDERLINE-SMOTE example



SMOTE: numerical example

$$\text{New sample} = \text{original sample} - \text{factor} * (\text{original sample} - \text{neighbour})$$



$$X_{\text{ori}} = (4, 7)$$

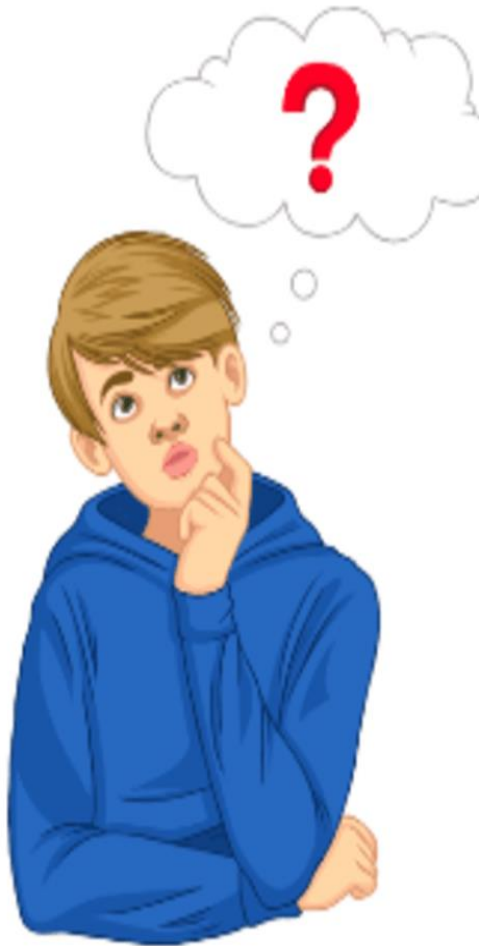
$$X_{\text{neig}} = (6, 8)$$

$$\text{New sample} = (4, 7) - 0.8 * ((4, 7) - (6, 8))$$

$$\text{New sample} = (4, 7) - 0.8 * ((-2, -1))$$

$$\text{New sample} = (4, 7) - ((-1.6, -0.8))$$

$$\text{New sample} = (5.6, 7.8)$$



```
from imblearn.over_sampling import SMOTE
```

```
from collections import Counter
```

```
counter = Counter(y_train)
```

```
print('Before', counter)
```

```
# oversampling the train dataset using SMOTE
```

```
smt = SMOTE()
```

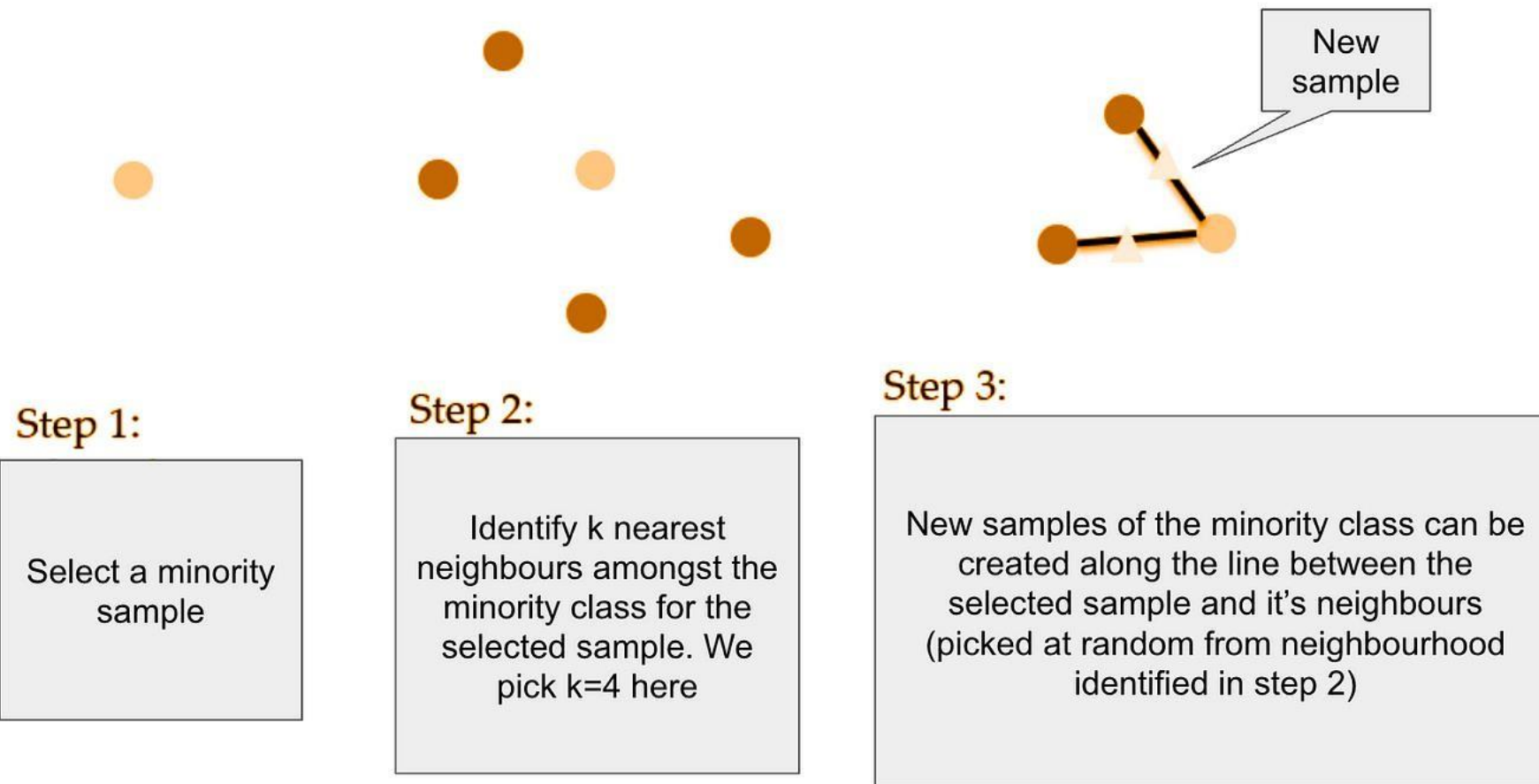
```
X_train_sm, y_train_sm = smt.fit_resample(X_train, y_train)
```

```
counter = Counter(y_train_sm)
```

```
print('After', counter)
```

ADASYN (Adaptive Synthetic Sampling):

based on the idea of adaptively generating minority data samples according to their distributions: more synthetic data is generated for minority class samples that are harder to learn compared to those minority samples that are easier to learn.



ADASYN

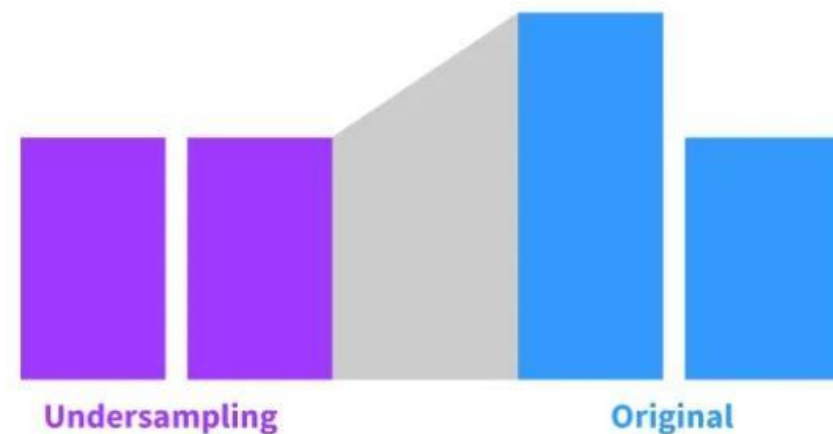
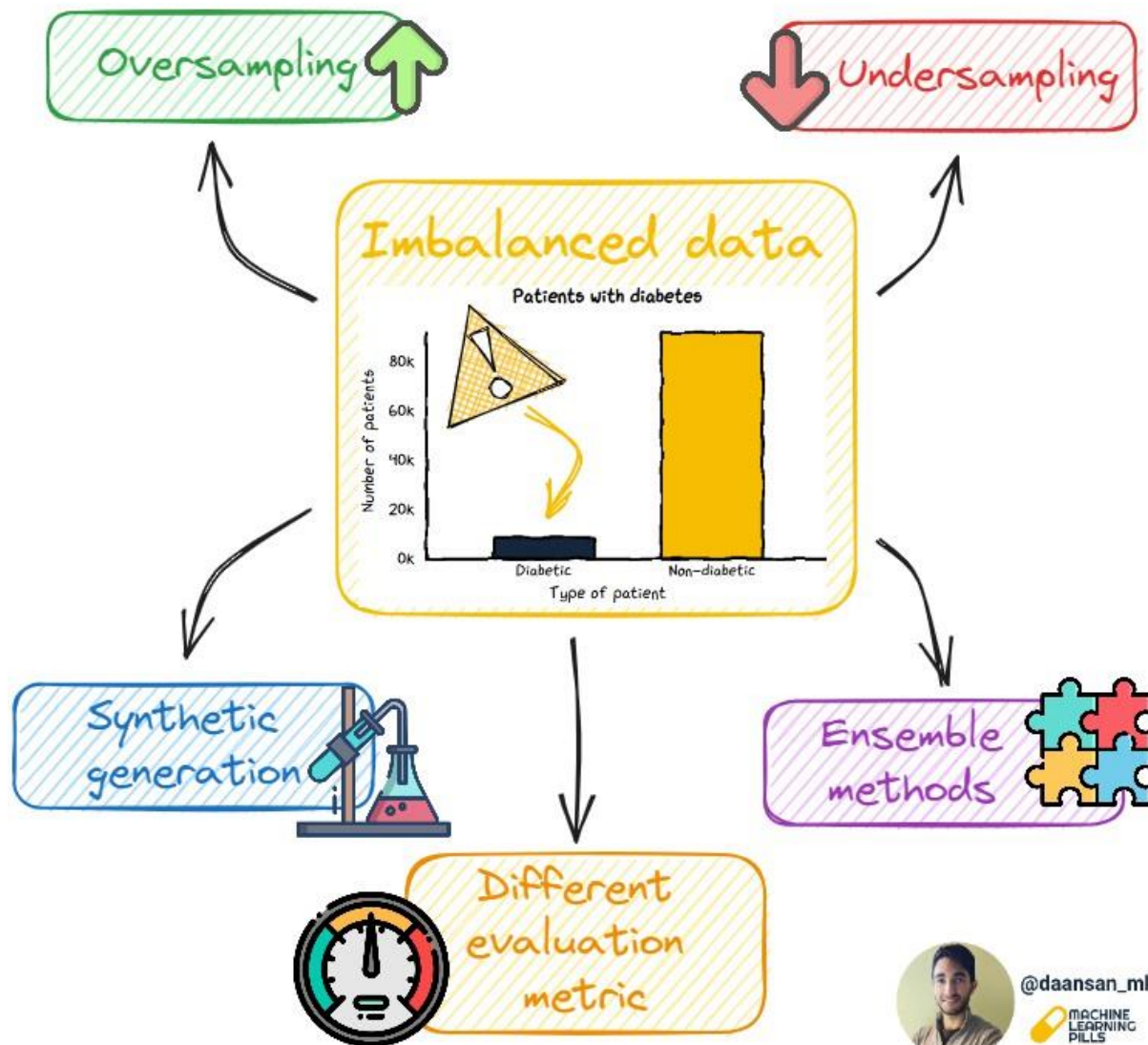
- Focuses more on “harder-to-learn” samples (those near the decision boundary).
- Steps:
 1. For each minority sample, count how many of its k-NN are majority class.
 - If many neighbors are majority → it’s a *hard* sample.
 - If few neighbors are majority → it’s an *easy* sample.
 2. Assign a weight r_i proportional to this difficulty.
 3. Decide how many synthetic samples to generate per point:

$$g_i = \frac{r_i}{\sum r_j} \cdot G$$

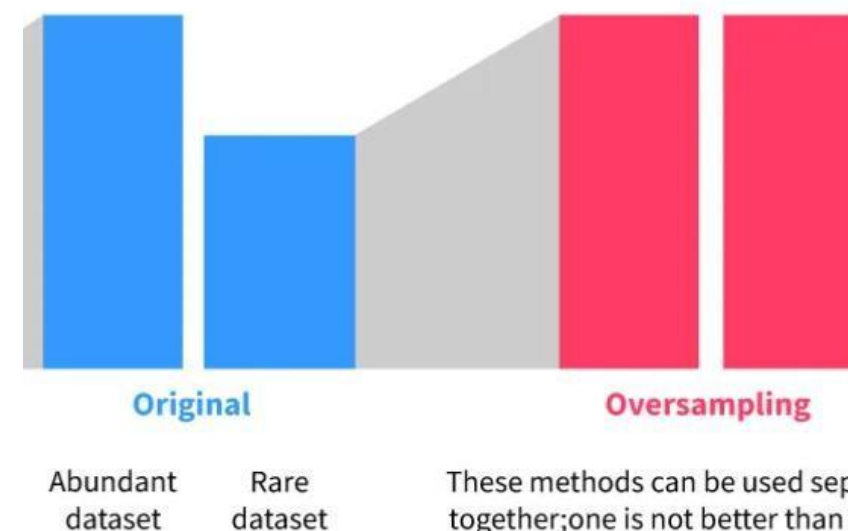
where G = total number of synthetic samples needed.

4. Generate synthetic samples the same way as SMOTE (interpolation), but biased toward *harder* samples.

How to address imbalanced data?



In undersampling, we pull all the rare events while pulling a sample of the abundant events in order to equalize the datasets.



These methods can be used separately or together; one is not better than the other. Which method a data scientist uses depends on the dataset and analysis.



@daansan_ml
MACHINE LEARNING PILLS

Data Pre-Processing

Data Cleaning

- Handling Missing Value
- Handling Outliers

Data Transformation

- Feature Scaling
- Feature Encoding
- Feature Engineering

Handling Imbalanced Data

- Under Sampling
- Over Sampling

Data Reduction

- Dimensionality Reduction
- Instance Reduction

Data Splitting

Data Pre-Processing: Data Reduction

- Data reduction in data preprocessing techniques refers to the process of reducing the volume but producing the same or similar analytical results.
- The goal is to simplify the dataset while retaining its essential characteristics, making it more manageable, efficient, and suitable for analysis or modeling.
- Data reduction techniques are particularly useful when dealing with large datasets or datasets with many features.
- There are two main types of data reduction:



Dimensionality Reduction: Reduce the number of input features while preserving important information.

Instance/Numerosity Reduction: Reduce the number of rows without losing information.

Data Reduction – Dimensionality Reduction

Dimensionality Reduction: Reduce the number of input features while preserving important information.

- **Principal Component Analysis (PCA):** PCA is a technique that transforms the original features into a new set of uncorrelated features called principal components. It allows for reducing the dataset's dimensionality while retaining as much variance as possible.
- **t-distributed Stochastic Neighbor Embedding (t-SNE):** t-SNE is a nonlinear dimensionality reduction technique that is particularly effective for visualizing high-dimensional data in lower-dimensional spaces.
- **Linear Discriminant Analysis (LDA):** LDA is a supervised dimensionality reduction technique that aims to maximize the separation between different classes in the dataset.

Data Reduction – Dimensionality Reduction

Principle Component Analysis

Before Dimensionality Reduction

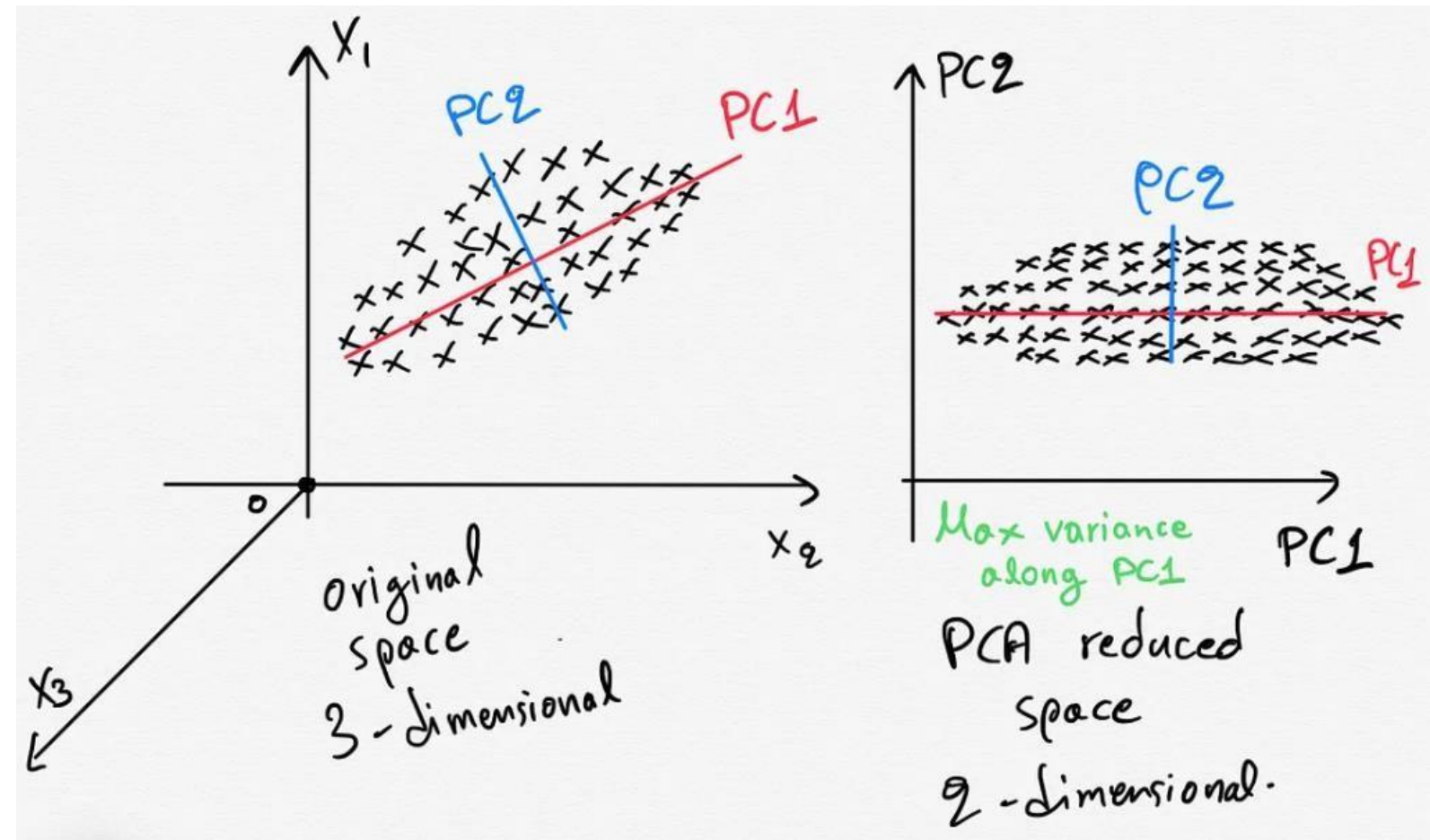
Product ID	Price	Number of Sales	Customer Reviews	Stock Quantity
101	20	200	4.5	50
102	15	150	4	40
103	30	120	4.8	30
104	25	180	4.2	20
105	10	300	3.8	60



After Dimensionality Reduction

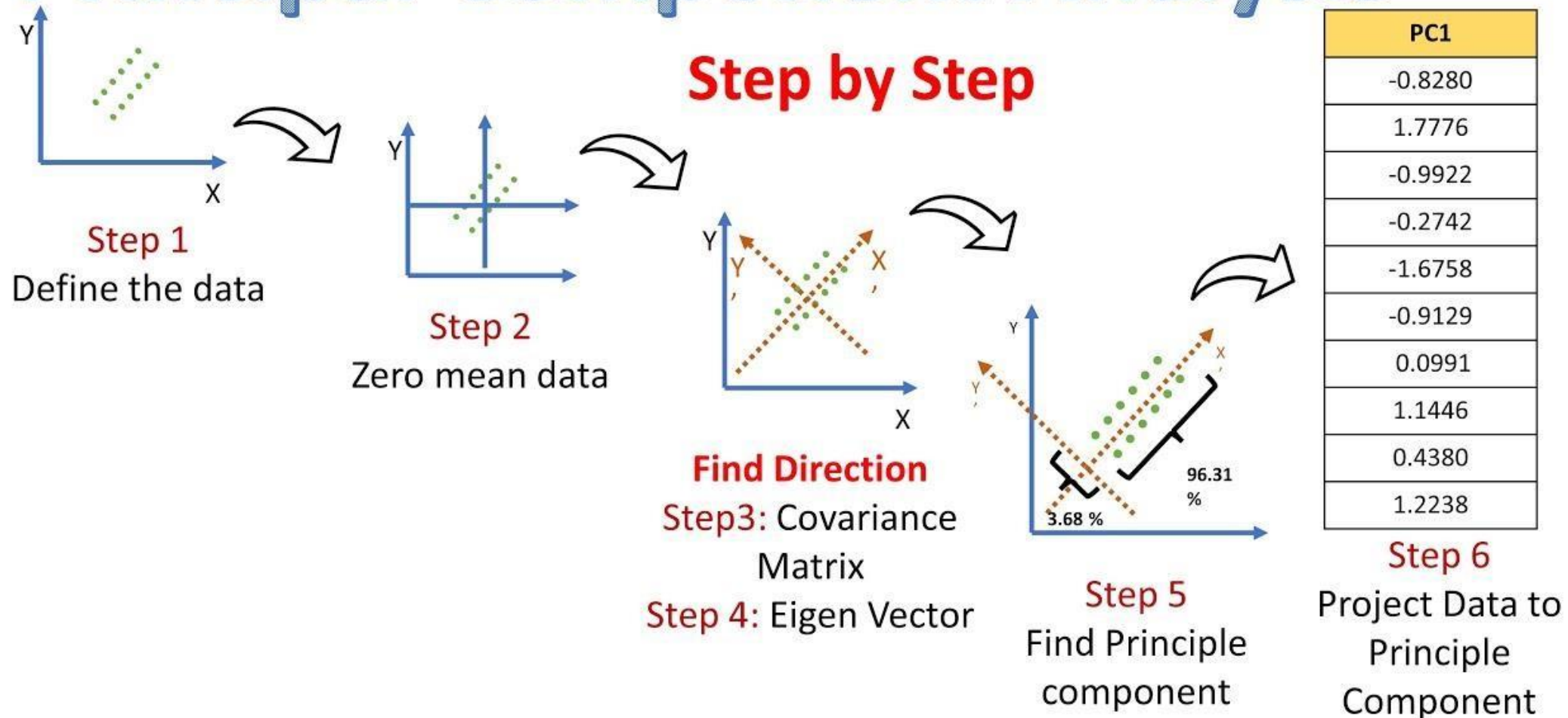
Product ID	Principal Component 1	Principal Component 2
101	2.3	-1.2
102	1.8	-0.9
103	3	-1.5
104	2.6	-1.1
105	1.2	-0.7

- A powerful tool for analyzing data in finding the patterns in the data (Feature extraction) as in the **name “Principal Component”** means major or maximum information.
- PCA is an **unsupervised linear dimensionality reduction algorithm** to find a more meaningful basis or coordinate system for our data and works based on **covariance matrix** to find the strongest features of your samples.



How To Compute

Principal Component Analysis



Convert the Given
data into 3
principle
components. If
n_components=3

Normally applied
to X_Axis

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=3)
```

```
scaler = StandardScaler()
```

```
X_train = pca.fit_transform(X_train)
```

```
X_test = pca.transform(X_test)
```

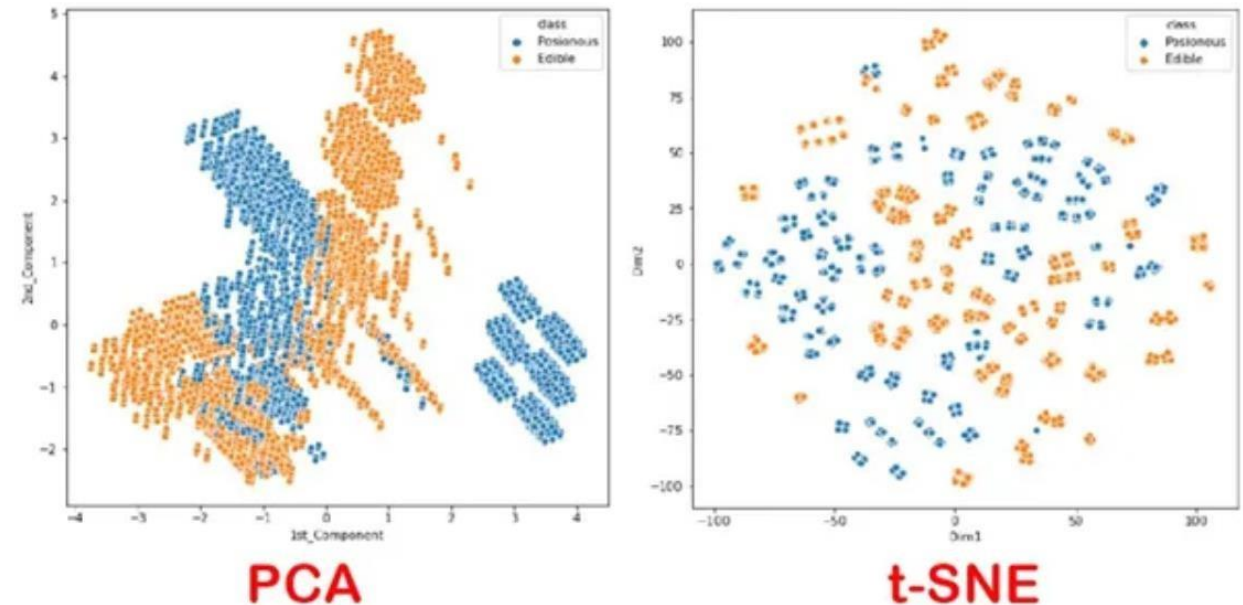
```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Data Reduction – Dimensionality Reduction

t-distributed Stochastic Neighbor Embedding (t-SNE)

- **Unsupervised Learning** used to reduce the dimensionality of the dataset.
- It's a **randomized non-linear method to map** the high dimensional data to the lower dimensional.
- t-SNE has a quadratic time and space complexity in the number of data points, as it considers the neighborhood also.



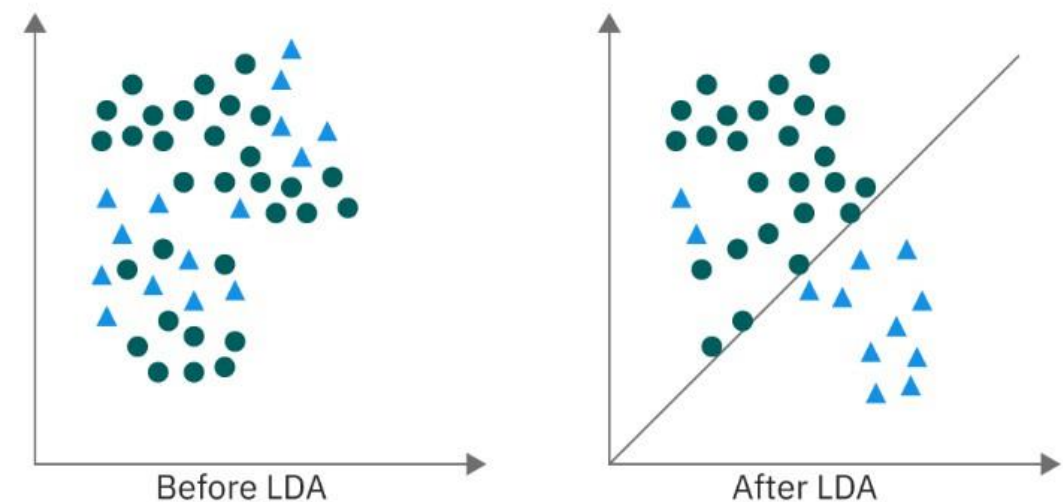
```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2)
X_tsne = tsne.fit_transform(X_std)
```

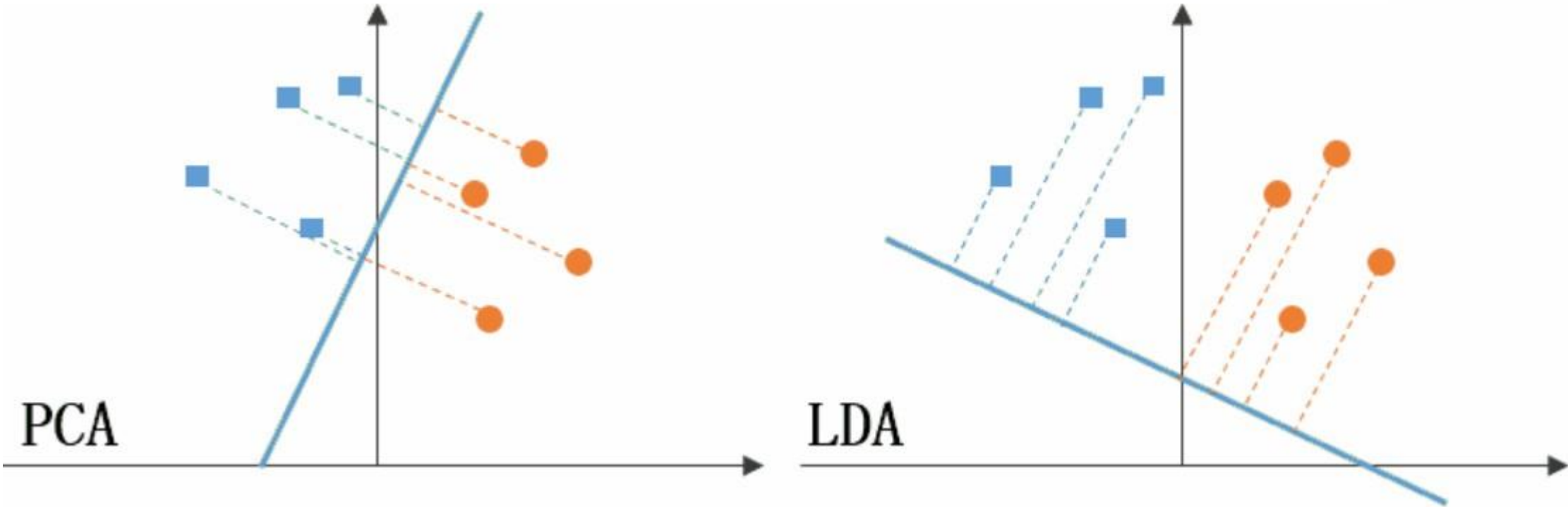
Data Reduction – Dimensionality Reduction

Linear Discriminant Analysis (LDA):

- **Supervised machine learning** to solve multi-class classification problems.
- LDA achieves this by using the X–Y axis to create a new axis, separating the different classes with a straight line and projecting data onto the new axis.
- To create this new axis and reduce dimensionality, LDA follows these criteria:
 - Maximize the distance between the means of two classes.
 - Minimize the variance within individual classes.



	PCA	LDA
Type of Transformation	Linear	Linear
Supervised vs Un-supervised	Un-supervised	Supervised
Relationship	Relationship between independent variables	Relationship between dependent and independent Variable
Objective	Capture the variability by finding Principal Components	Class separation by identifying a lower dimensional space which has better discriminatory power



Data Reduction – Instance/Numerosity Reduction

Reduce the number of rows without losing information.

- **Sampling:** Random or systematic sampling can select a subset of instances from the dataset. This can help reduce the dataset size while maintaining its statistical properties.
- **Clustering:** Clustering techniques, such as k-means clustering, can be applied to group similar instances together. Instead of using all instances, representative centroids or cluster prototypes can be used, reducing the number of instances while preserving the overall structure.
- **Data Binning or Discretization:** Continuous data can be converted into discrete intervals, reducing the number of unique values and simplifying the dataset.
- **Outlier Removal:** Instances that are considered outliers can be removed from the dataset. Outliers may introduce noise and distort the analysis.

Data Reduction – Instance Reduction

Data Binning or Discretization

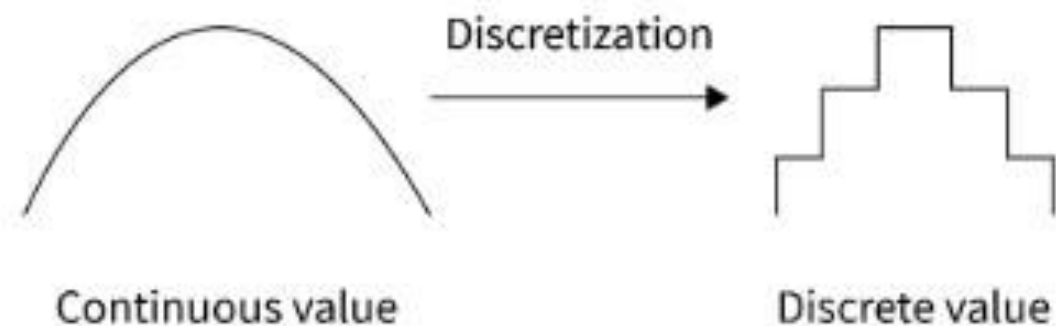
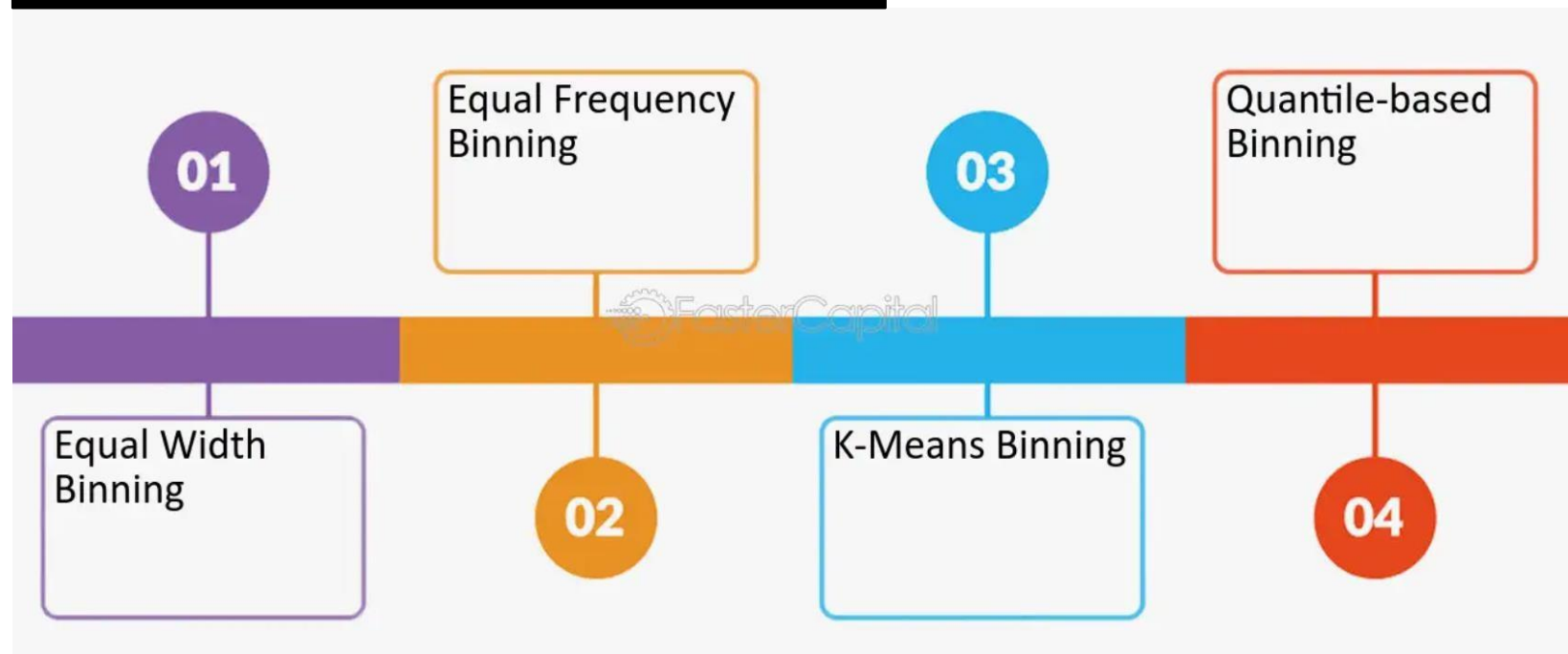
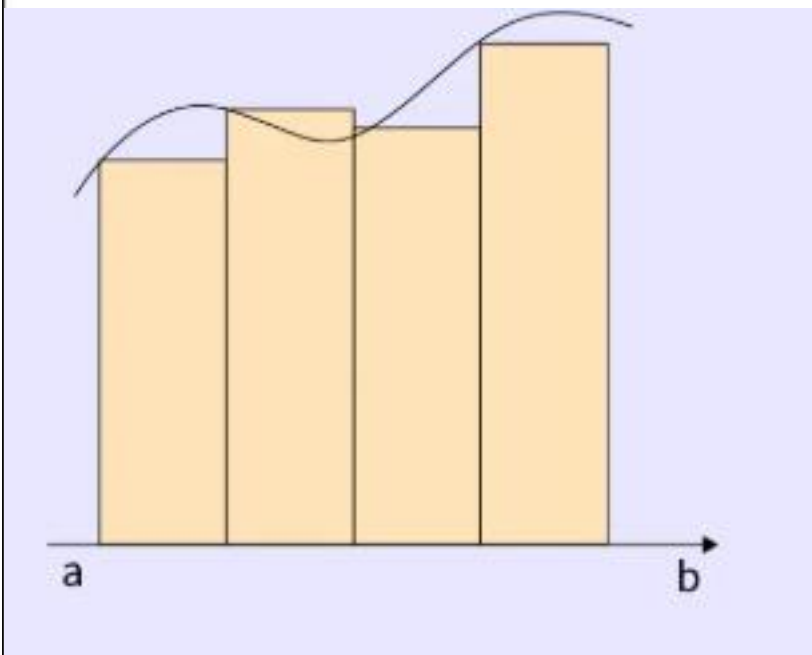
It involves dividing a continuous variable into a set of smaller intervals or bins and replacing the original values with the corresponding bin labels. Binning can be applied to both numerical and categorical variables, and its primary purpose is to simplify the data and make it more manageable for analysis.

Data That Often Benefits from Binning:

- Continuous variables with wide ranges
- Skewed distributions
- Variables with outliers
- High-cardinality numerical data



Types of Data Binning or Discretization



Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Binning

- ▣ First sort data and partition into (equal-frequency) bins
- ▣ Then one can **smooth by bin means**, **smooth by bin median**, **smooth by bin boundaries**, etc.

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15

Bin 2: 21, 21, 24

Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9

Bin 2: 22, 22, 22

Bin 3: 29, 29, 29

Smoothing by bin boundaries:

Bin 1: 4, 4, 15

Bin 2: 21, 21, 24

Bin 3: 25, 25, 34

Data Pre-Processing

Data Cleaning

- Handling Missing Value
- Handling Outliers

Data Transformation

- Feature Scaling
- Feature Encoding
- Feature Engineering

Handling Imbalanced Data

- Under Sampling
- Over Sampling

Data Reduction

- Dimensionality Reduction
- Instance Reduction

Data Splitting

Data Pre-Processing: Data Splitting

- Splitting the total dataset
- **Train Data(Training)**
 - Data consists of input features and their corresponding output labels
 - Use for model learning patterns from this data and uses them to make predictions on new, unseen data.
- **Test Data(Testing)**
 - Used to evaluate the final performance of our model after training and tuning.
 - Data should be completely separate from the train and validation data to avoid any data leakage.

Type 1: Random Test Train Split

- where we randomly divide our dataset into a training set and a test set.
- `train_test_split()`

Example 1

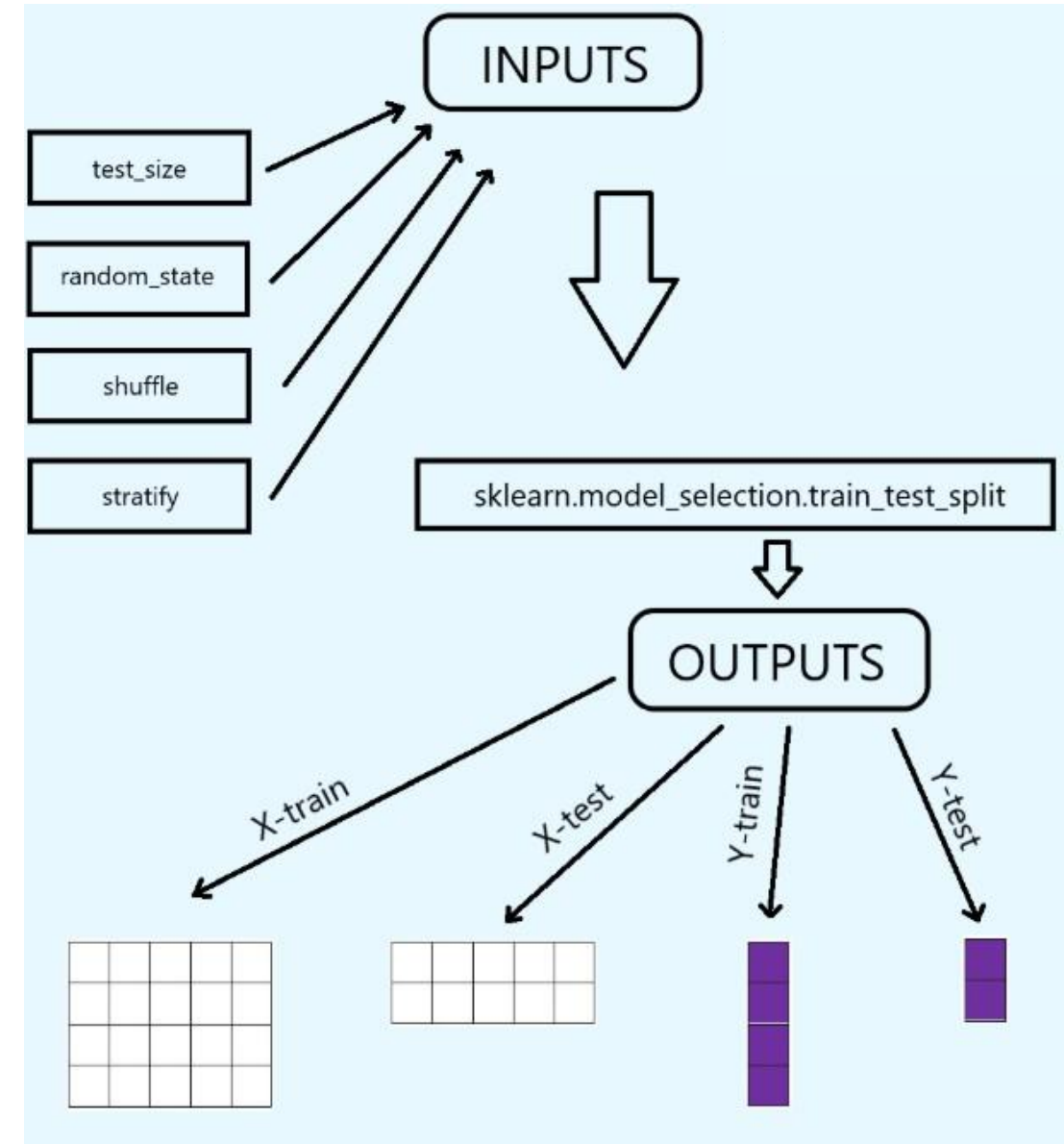
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    train_size=0.67,
                                                    random_state=42)

print(f"Train labels:\n{y_train}")
print(f"Test labels:\n{y_test}")
```

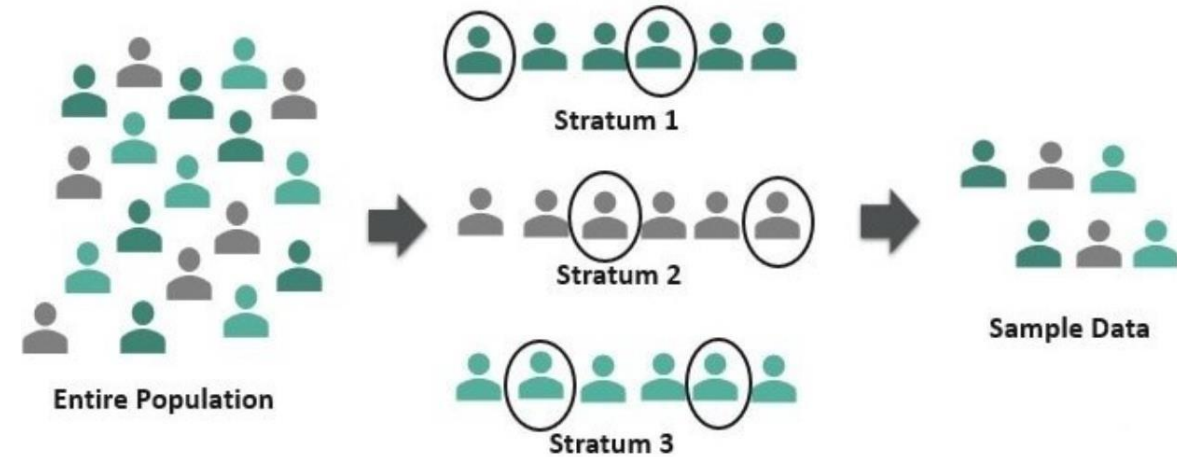
Example 2

```
# Split dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2, random_state=42)
```



Type 2: Stratified Sampling

- Ensures that the proportion of each class in the training and test sets is the same as that in the original dataset



- Example 1

```
X_test, X_val, y_test, y_val = train_test_split(X_test, y_test,
                                              train_size=0.5,
                                              random_state=42,
                                              stratify=y_test)
```

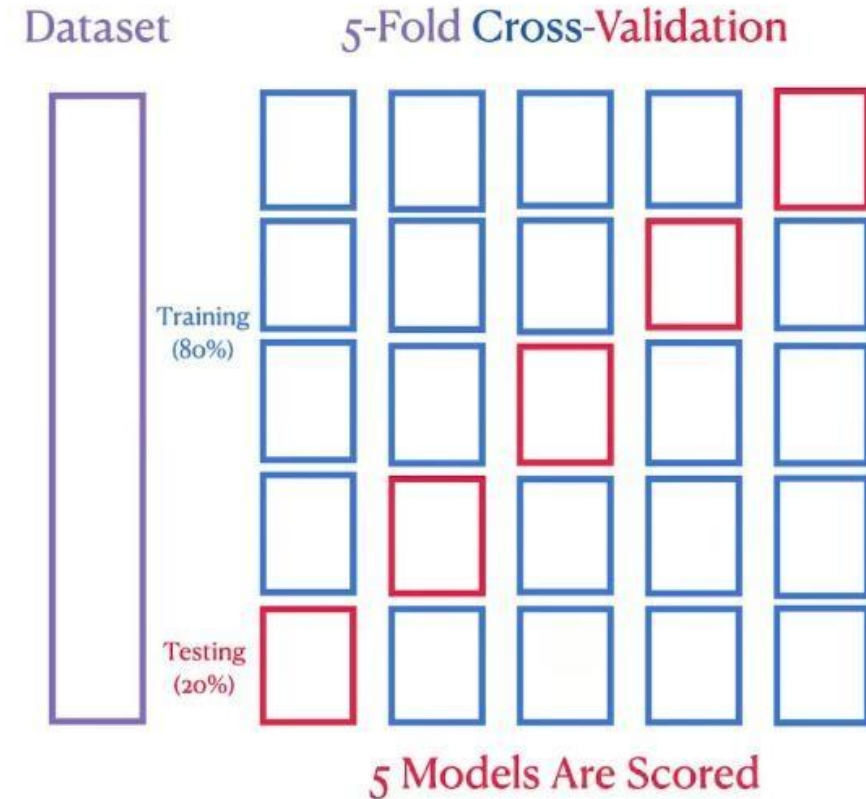
- Example 2

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8,
                                                  random_state=None,
                                                  shuffle=True, stratify=y)

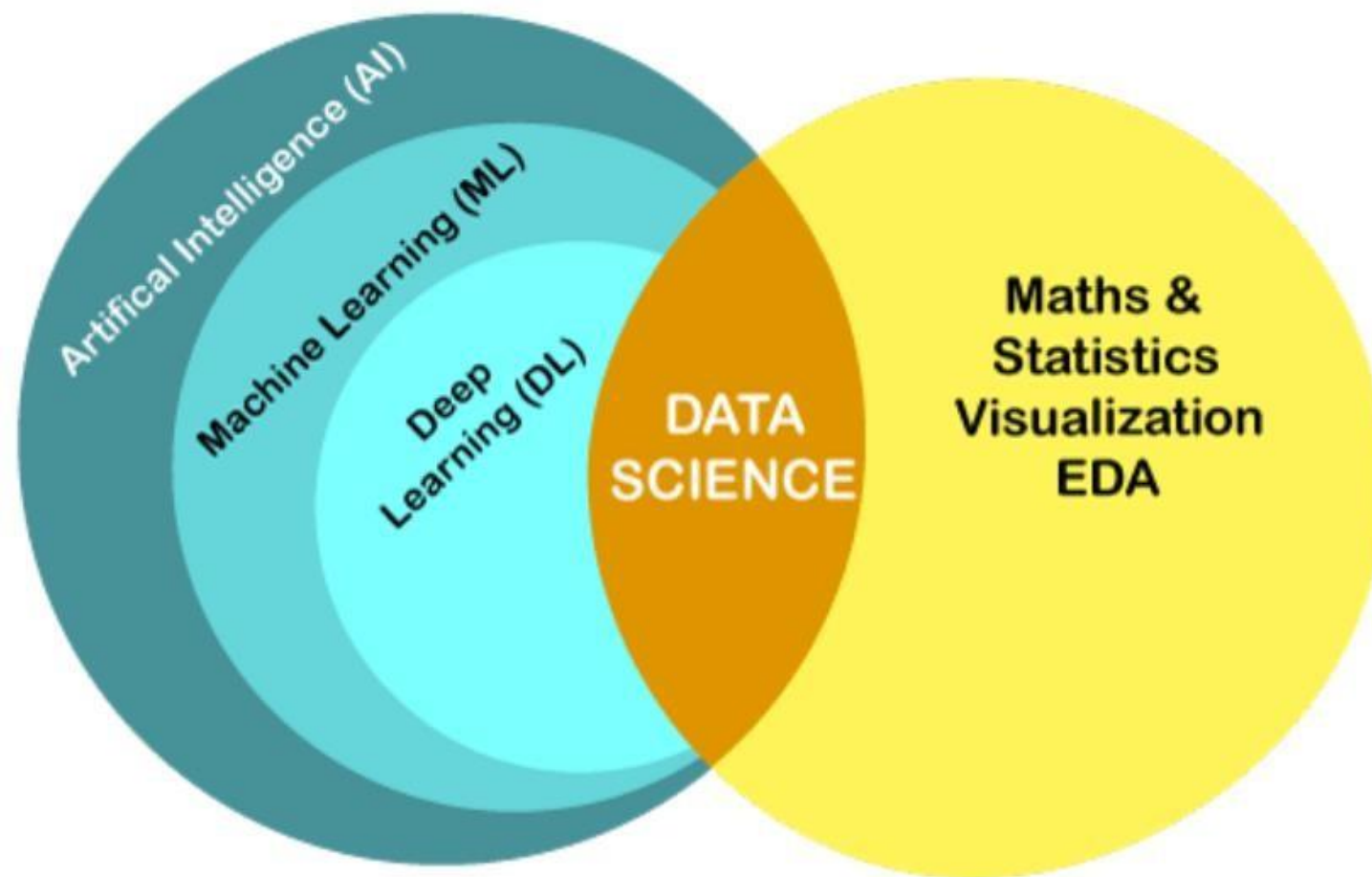
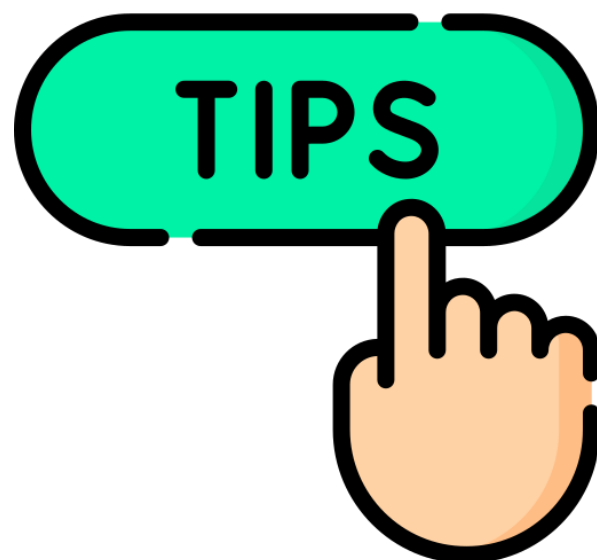
print(y_train.value_counts())
print(y_test.value_counts())
```

Type 3: CrossFold Data

- As data points for training and testing sets change randomly, this will result in fluctuating accuracy. To prevent this, the concept of cross-validation is used.
- Common cross-validation techniques include:
 - K-fold cross-validation**
 - Stratified cross-validation** (Primarily classification with grouped data)



```
from sklearn.model_selection import KFold
k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)
```



Data Preprocessing? — Major Tasks

□ Data cleaning

- Handle missing data, smooth noisy data, identify or remove outliers, and resolve inconsistencies

□ Data integration

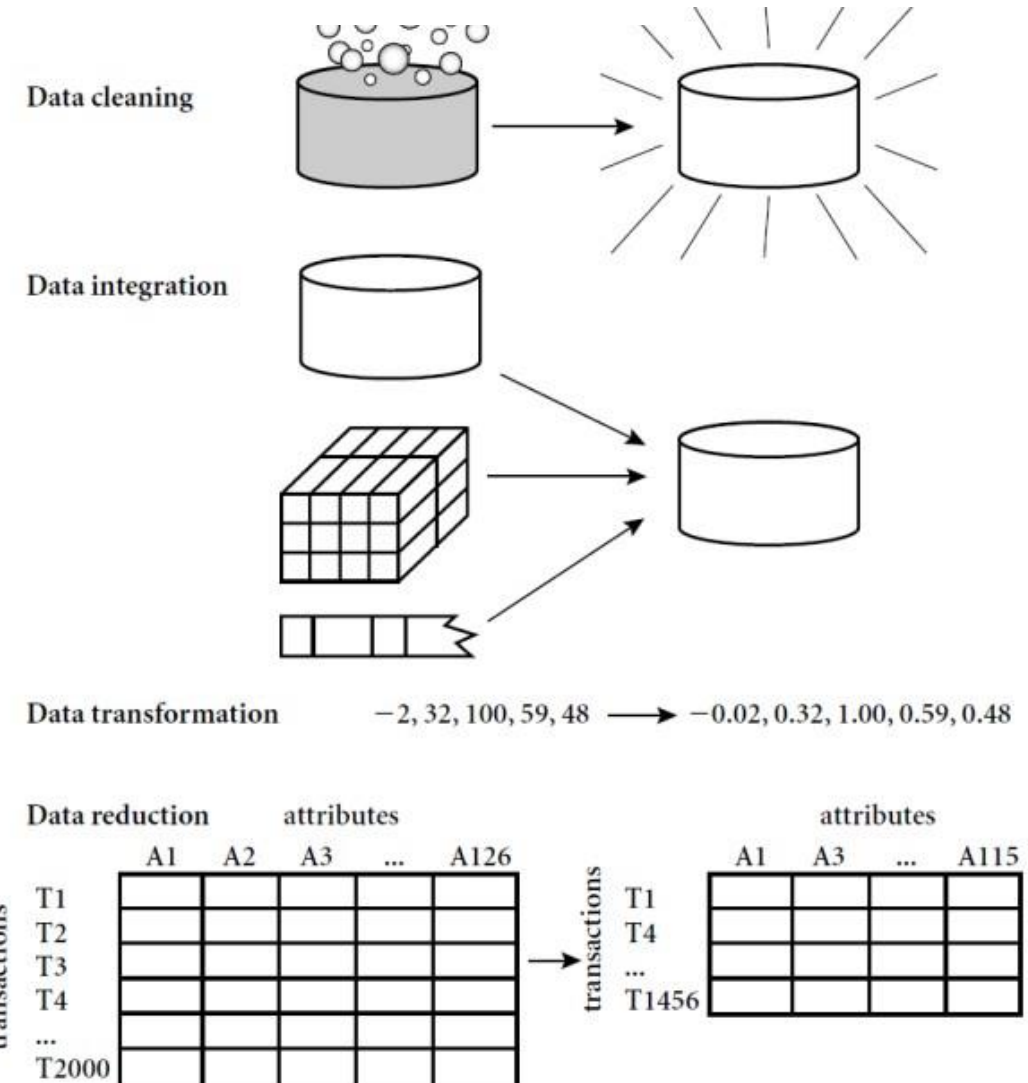
- Integration of multiple databases, data cubes, or files

□ Data reduction

- Dimensionality reduction
- Numerosity reduction
- Data compression

□ Data transformation and data discretization

- Normalization
- Concept hierarchy generation



Thank You